

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

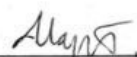
Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

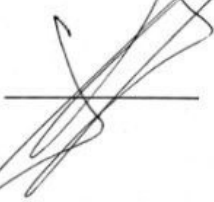
Отчет о выполнении курсовой работы
по дисциплине «Теоретические основы баз данных»

Реализация базы данных для предметной области
«Функционирование работы бассейна на коммерческой основе»

Студент,
группы 5130201/30003

 Мартыненко А. П.

Преподаватель,
доц., кнд. доц.

 Попов С. Г.

« 30 » 04 2025г.

1/3 19.03.25
2/3 26.03.25
3/3 30.04.25
Санкт-Петербург, 2025

Реферат

Предметной областью является функционирование работы бассейна на коммерческой основе, включая учёт посещаемости и финансов. Проблема заключается в необходимости систематизации данных о клиентах, сотрудниках, посещениях и платежах для эффективного управления учреждением. В результате выполнения работы была создана база данных, содержащая информацию о посетителях, персонале, посещениях, услугах и оплате. Были выполнены 8 запросов и построены 3 гистограммы для наглядного представления информации. База данных и запросы были написаны на языке SQL, для создания базы данных использовалась командная строка, для выполнения запросов использовалась среда MySQL Workbench 8.0 CE. Разработанная система может применяться в коммерческих бассейнах для автоматизации учёта посещений и финансов.

Содержание

Введение	5
1 Постановка задачи	6
2 Исследование процессов работы бассейна и системы абонентов	7
2.1 Описание предметной области	7
2.2 Цели создания базы данных	9
2.3 Атрибуты сущностей и ролей	10
2.4 Построение ER-диаграммы	11
2.5 Чтение ER-диаграммы	12
2.6 Схема объектов	12
3 Проектирование	13
3.1 Таблицы с описанием таблиц баз данных	13
3.2 Лингвистическое описание схемы	17
3.3 Графическое описание схемы базы данных на русском языке	19
3.4 Графическое описание схемы базы данных на английском языке	20
3.5 Создание базы данных и заполнение значениями	21
4 Программирование	22
4.1 Запрос №1.1	23
4.2 Запрос №1.2	25
4.3 Запрос №2	27
4.4 Запрос №3.1	28
4.5 Запрос №3.2	30
4.6 Гистограмма распределения для запросов 3.1 и 3.2	32
4.7 Запрос №4	33
4.8 Запрос №5	37
4.9 Запрос №6	38
4.10 Запрос №7	40
4.11 Запрос №8.1	42
4.12 Запрос №8.2	43
Заключение	45
Список использованных источников	46

Приложение А «Исходный код программы генерации схемы базы данных»	47
Приложение Б «Исходный код программы заполнения базы данных тестовыми данными»	51
Приложение 1. Список разрешенных предметов	62

Введение

Работа с большими объемами данных требует системного подхода к их организации, хранению и быстрому доступу. При ручной обработке данных возрастает риск ошибок, утерь информации, а также значительно увеличивается время на поиск и анализ необходимых сведений.

Система баз данных - это компьютеризированная система хранения однотипных записей. Саму базу данных можно рассматривать как подобие электронной картотеки, то есть хранилище или контейнер для некоторого набора данных, занесенных в компьютер. Она хранит информацию структурированно и упрощает её обработку, обеспечивая целостность, согласованность и защиту данных.

Внедрение баз данных позволяет ускорить выполнение рутинных операций, минимизировать дублирование и обеспечить одновременный доступ нескольких пользователей к массиву данных. Это может быть полезно для эффективного управления бассейном и планирования бюджета организации.

Взаимодействие с базами данных, а именно редактирование, добавление и удаление данных, написание и выполнение запросов осуществляется с помощью языка SQL. Этот язык позволяет получать выборки по заданным критериям для детального анализа, а также обеспечивает быстрый доступ к информации даже при ее постоянном обновлении и увеличении объема.

1 Постановка задачи

Исходные данные: предметная область, тексты формулировок запросов к базе данных на русском языке.

Цель: реализовать базу данных для предметной области «Функционирование работы бассейна на коммерческой основе».

Задачи:

1. Описание предметной области и формулировка целей создания базы данных.
2. Создание ER-диаграммы.
3. Проектирование базы данных, создание графического описания.
4. Создание базы данных и заполнение 300-400 тысячами значений.
5. Написание текстов 8 запросов на языке SQL и генерация 3 гистограмм.

2 Исследование процессов работы бассейна и системы абонементов

2.1 Описание предметной области

Плавательный бассейн – это сооружение, предназначенное для занятий водными видами спорта, в основном плаванием. [1] Бассейн заполняется хлорированной водой и имеет длину 25 или 50 метров. Бассейн разделен на 6 дорожек, комфортная вместимость каждой из которых 8 человек, в случае детей – до 12 человек. На скоростных дорожках установлены предупреждения об этом, движение совершается по правой стороне, за исключением сеансов аквааэробики. Для посещения бассейна необходим действующий абонемент. При посещении можно арендовать полотенце, очки, шапочку для плавания. Инвентарь ежедневно дезинфицируется. Бассейн бесплатно предоставляет посетителям дополнительный инвентарь для занятий: доски, колобашки, нудлы, ласты и лопатки. Для посетителей, которые чувствуют себя неуверенно в воде, предусмотрены спасательные жилеты и надувные нарукавники, которые выдаются по запросу. Также возможно использование своего инвентаря, который не мешает окружающим и входит в список разрешенных предметов. Список разрешенных предметов находится на информационной доске и в приложении 1. В бассейне есть правила, которые необходимо соблюдать всем посетителям для комфортного и безопасного пребывания. Они расположены на информационной доске, а также в помещении чаши бассейна. Во время тренировки или сеанса в помещении чаши бассейна находится спасатель, который контролирует исполнение этих правил, а также следит за безопасностью посетителей.

Абонемент – это бумажный документ, который фиксирует оплату одного или нескольких сеансов плавания и предоставляет право на посещение бассейна указанное число раз. Абонементом можно воспользоваться в течение полугода с момента покупки. Для продажи абонемента необходимы личные данные посетителя: ФИО, возраст, если младше 18 лет – ФИО родителя или ответственного, наличие справки от дерматолога полученной не раньше, чем за полгода [3]. Абонементы делятся на детские и взрослые. В детские абонементы входит занятия под руководством тренера в группе не более 10 человек по определенному расписанию. Детские абонементы делятся по ценовому сегменту на «игровое плавание», «обучение плаванию», «спортивное плавание», «оздоровительное плавание». Детские тренировки имеют фиксированное время и продолжительность, которые зависят от типа тренировки и возраста ребенка. Группы

формируются из детей с разницей в возрасте не более двух лет. Взрослые абонементы разделены на открытое посещение и с сеткой расписания. Открытое посещение подразумевает посещение бассейна в интервал 15:00-18:00 в будние дни и 11:00-18:00 в выходные дни, время нахождения в бассейне не фиксировано. Абонементы с сеткой расписания разделены на типы «свободное плавание», «аквааэробика», «обучение плаванию». В зависимости от выбранного типа абонемента формируется расписание по часам и дням недели, при этом часы фиксированы, день недели можно выбирать, исходя из графика посетителя. У каждого абонемента есть особое свойство – уникальный номер. На абонементе написано ФИО посетителя, количество посещений, срок годности и тариф. [2]

С расписанием сеансов можно ознакомиться в кассе бассейна, на информационной доске, а также на сайте организации. Расписание обновляется раз в квартал, объявления о соревнованиях, санитарных днях или отмене занятий добавляются по мере поступления информации.

В продаже абонемента участвует посетитель или ответственный за него человек в случае детских абонементов и администратор. При продаже нескольких абонементов возможна скидка, также действуют льготные билеты для пенсионеров, детей и студентов при наличии подтверждающих документов. Стоимость абонементов зависит от количества посещений, выбранного типа занятий и наличия/отсутствия льгот. При покупке абонемента оформляется договор об оказании услуг между организацией и посетителем, подписывается согласие на обработку персональных данных. По запросу посетителю могут быть предоставлены необходимые лицензии и сертификаты, а также предоставлена дополнительная информация, например, какую группу лучше выбрать, исходя из запросов посетителя.

При первом посещении по абонементу проводится инструктаж по технике безопасности, по результатам которого посетитель расписывается в знании правил поведения в бассейне. При каждом посещении проводится медицинская проверка спасателем. По результатам ее прохождения на абонементе делается отметка о посещении, а именно дата посещения и подпись администратора. Во время сеанса абонемент остается на стойке администратора, в обмен на него выдается ключ от шкафчика для вещей в раздевалке. По окончании сеанса ключ необходимо вернуть на стойку и забрать абонемент.

В случае истечения срока годности абонемента при наличии неиспользованных посещений посетитель может израсходовать эти сеансы при повторном предъявлении медицинской справки. Отказ от абонемента с возвратом средств возможен только по медицинским противопо-

казаниям, при этом деньги возвращаются только за неиспользованные сеансы. Администрация оставляет за собой право аннулирования абонемента. Оно возможно в случаях, когда пользователь неоднократно нарушает правила поведения в бассейне, подвергая опасности свою жизнь и жизни окружающих. При этом денежные средства за оставшиеся сеансы не возвращаются. Также медперсонал - спасатель может не допустить посетителя к занятиям из-за плохого самочувствия или открытых ран.

В случае получения травмы посетителю оказывается первая медицинская помощь, которую проводит квалифицированный спасатель. Если несчастный случай оказывается страховым, например, травмы из-за падения на мокром полу, из-за повреждения оборудования, администрация, спасатель и посетитель составляют акт для дальнейшего обращения в страховую компанию. При этом данное посещение может быть компенсировано, о чем в абонементе делается соответствующая запись. Перечень страховых случаев представлен на информационной доске. Если несчастный случай не является страховым, как игнорирование инструкций спасателя, нарушение правил поведения, то ответственность несет посетитель или ответственный за него совершеннолетний. Администрация бассейна фиксирует каждый несчастный случай с указанием его типа и полной информации, включая показания свидетелей и спасателя, а также характера травмы. Для предотвращения несчастных случаев, в бассейне техническим персоналом проводится ежедневная уборка, замеры показателей воды: уровень pH, хлора, чистоты, а также ежемесячно устраивается санитарный день с полной очисткой чаши бассейна.

Для получения обратной связи посетителям предлагается периодическое прохождение анонимных опросов, а также наличие книги жалоб и предложений. Также фиксируется количество проданных абонементов по типам сеансов, наиболее популярные часы занятий.

2.2 Цели создания базы данных

1. Учет продаж абонементов в бассейн.
2. Контроль количества людей в бассейне в определенный момент.
3. Определение наиболее востребованных типов и времени сеансов.
4. Хранение расписания сеансов.

2.3 Атрибуты сущностей и ролей

Сущности:

Тип абонемента - шаблон абонемента, содержащий в себе тип выбранных занятий и их дни и время, который используется для формирования абонемента.

Абонемент - документ, который предоставляет предъявителю право на посещение сеанса бассейна определенное количество раз.

Сетка расписания - документ, устанавливающий ограничения по времени для сеансов в зависимости от выходных или будних дней.

Посещение - факт прихода посетителя на сеанс.

Покупка - факт покупки посетителем или ответственным за него лицом одного или нескольких абонементов.

Прейскурант - документ, утверждающий цену за каждый тип абонемента.

Роли:

Администратор бассейна - лицо, ответственное за продажу абонементов, учет посещений и управление сеансами в бассейне.

Посетитель - лицо, посещающее сеанс в бассейне.

Ответственный - лицо, несущее за несовершеннолетнего (ребенка), который может покупать за него абонемент.

Атрибуты сущностей и ролей:

Тип абонемента. Атрибуты: время сеанса, дни сеанса, тип абонемента, указание детский/взрослый.

Абонемент. Атрибуты: тип абонемента, дата начала, дата окончания, количество посещений.

Сетка расписания. Атрибуты: будний/выходной день, время сеанса.

Посещение. Атрибуты: дата посещения, день недели, сетка расписания.

Покупка. Атрибуты: дата покупки, стоимость, фамилия администратора, фамилия покупателя.

Прейскурант. Атрибуты: тип абонемента, стоимость, объем занятий, уровень нагрузки.

Администратор бассейна. Атрибуты: фамилия, имя, отчество администратора, опыт работы.

Посетитель. Атрибуты: фамилия, имя, отчество, дата рождения, ответственный, наличие медицинской справки, контактный телефон.

2.4 Построение ER-диаграммы

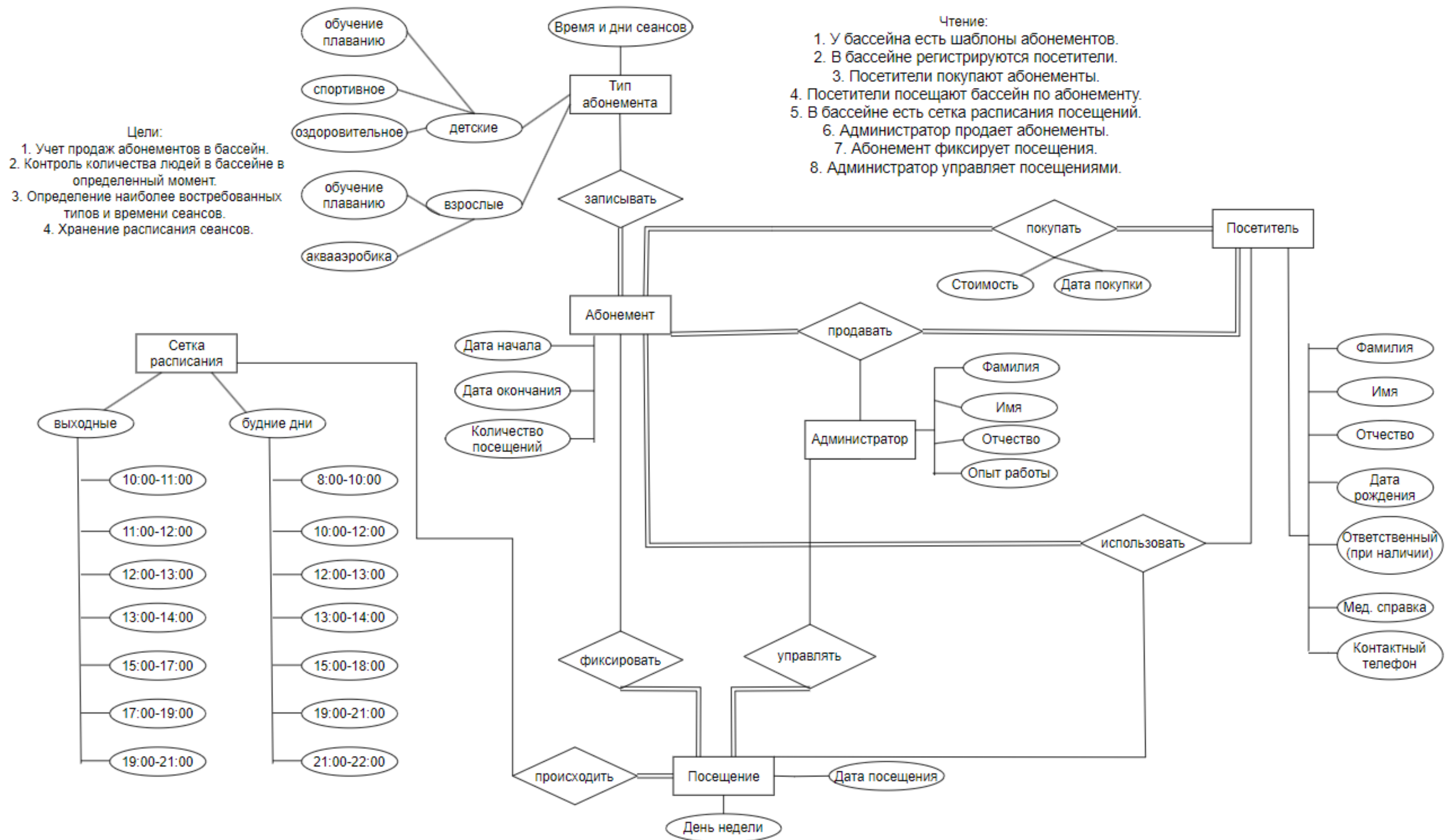


Рис. 1: Схема ER-диаграммы

2.5 Чтение ER-диаграммы

ER-диаграмма представлена на рисунке 1.

1. У бассейна есть шаблоны абонементов.
2. В бассейне регистрируются посетители.
3. Посетители покупают абонементы.
4. Посетители посещают бассейн по абонементу.
5. В бассейне есть сетка расписания посещений.
6. Администратор продает абонементы.
7. Абонемент фиксирует посещения.
8. Администратор управляет посещениями.

2.6 Схема объектов

Схема объектов представлена на рисунке 2.

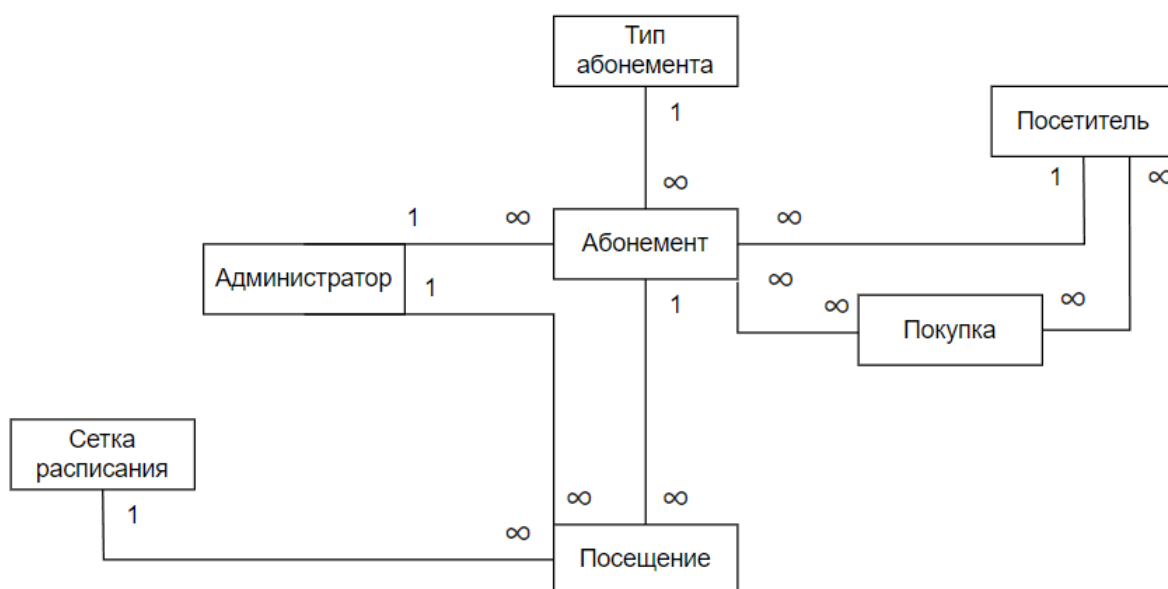


Рис. 2. Схема объектов базы данных

3 Проектирование

3.1 Таблицы с описанием таблиц баз данных

В таблицах 1-14 представлены атрибуты таблиц базы данных. Каждая таблица с описанием содержит колонки: номер по порядку, имя атрибута на русском и английском языках, тип атрибута, при необходимости со значением размера, значение по умолчанию, признак ключа при наличии, для внешнего ключа ссылка на таблицу, примечание.

Ограничение в 17 символов для атрибута «Название вида абонемент» установлено потому, что это максимальное количество символов среди всех присутствующих в предметной области видов.

Ограничение в 40 символов для атрибута «Фамилия» установлено с учетом двойной самой длинной русской фамилии.

Ограничение в 15 символов для атрибута «Имя» установлено с учетом самого длинного русского имени.

Ограничение в 20 символов для атрибута «Отчество» установлено с учетом преобразования от атрибута имени с добавлением нескольких образующих символов.

Ограничение в 15 символов для атрибута «Телефон» установлено с возможностью добавления дополнительных символов к основным 11 цифрам номера телефона.

Ограничение в 30 символов для атрибута «Дни сеанса» установлено по максимальному количеству символов трех дней недели, так как один тип занятий не может идти более трех дней в неделю.

Ограничение в 8 символов для атрибута «Тип дня» установлено по максимальному количеству символов среди возможных значений: «будний» и «выходной».

Ограничение в 11 символов для атрибута «День недели» установлено по максимальному количеству символов среди названий дней недели.

Ограничение в 8 символов для атрибута «Возрастной тип» установлено по максимальному количеству символов среди возможных значений.

Ограничение в 7 символов для атрибута «Сложность» установлено по максимальному количеству символов среди возможных значений предметной области.

Таблица 1. Тип дня

Тип дня - Type of the day							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	День_id	id_day	INT	-	PK	-	NOT NULL
2	Тип дня	type_of_day	VARCHAR(8)	'будний'	-	-	NOT NULL

Таблица 2. День недели

День недели - Weekday							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	День_недели_id	id_weekday	INT	-	PK	-	NOT NULL
2	День недели	day_week	VARCHAR(11)	-	-	-	NOT NULL

Таблица 3. Уровень нагрузки

Уровень нагрузки - Skilltest							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	Уровень_нагрузки_id	id_skill	INT	-	PK	-	NOT NULL
2	Сложность	complexity	VARCHAR(7)	-	-	-	NOT NULL

Таблица 4. Категория абонемента

Категория абонемента - Category of pass							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	Категория_id	id_category	INT	-	PK	-	NOT NULL
2	Возрастной тип	type_age	VARCHAR(8)	-	-	-	NOT NULL

Таблица 5. Ответственный

Ответственный - Responsible							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	Ответственный_id	id_responsible	INT	-	PK	-	NOT NULL
2	Фамилия_ответственного	responsible_surname	VARCHAR(40)	-	-	-	NOT NULL
3	Имя_ответственного	responsible_name	VARCHAR(15)	-	-	-	NOT NULL
4	Отчество_ответственного	responsible_patronymic	VARCHAR(20)	NULL	-	-	NULL
5	Телефон_ответственного	responsible_phone	VARCHAR(15)	-	-	-	NOT NULL

Таблица 6. Администратор

Администратор - Admin							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	Администратор_id	id_admin	INT	-	PK	-	NOT NULL
2	Фамилия_администратора	admin_surname	VARCHAR(40)	-	-	-	NOT NULL
3	Имя_администратора	admin_name	VARCHAR(15)	-	-	-	NOT NULL
4	Отчество_администратора	admin_patronymic	VARCHAR(20)	NULL	-	-	NULL
5	Опыт_работы	admin_experience	INT	0	-	-	NOT NULL

Таблица 7. Вид абонемента

Вид абонемента - Kind of pass							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	Вид_абонемента_id	id_kind_of_pass	INT	-	PK	-	NOT NULL
2	Название_вида_абонемента	kind_name	VARCHAR(17)	-	-	-	NOT NULL
3	Категория_id	id_category	INT	-	FK	Category of pass	NOT NULL

Таблица 8. Сетка расписания

Сетка расписания - Schedule							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	Расписание_id	id_schedule	INT	-	PK	-	NOT NULL
2	День_id	id_day	INT	-	FK	Type of the day	NOT NULL
3	Начало_сеанса	start_time	TIME	-	-	-	NOT NULL
4	Конец_сеанса	end_time	TIME	-	-	-	NOT NULL

Таблица 9. Тип абонемента

Тип абонемента - Type of pass							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	Тип_абонемента_id	id_type_of_pass	INT	-	PK	-	NOT NULL
2	Расписание_id	id_schedule	INT	-	FK	Schedule	NOT NULL
3	Дни_сеанса	days_session	VARCHAR(30)	-	-	-	NOT NULL
4	Вид_абонемента_id	id_kind_of_pass	INT	-	FK	Kind of pass	NOT NULL

Таблица 13. Покупка

Покупка - Purchase							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	Покупка_id	id_purchase	INT	-	PK	-	NOT NULL
2	Дата_покупки	purchase_date	DATE	-	-	-	NOT NULL
3	Администратор_id	id_admin	INT	-	FK	Admin	NOT NULL
4	Абонемент_id	id_pass	INT	-	FK	Pass	NOT NULL
5	Посетитель_id	id_visitor	INT	-	FK	Visitor	NOT NULL
6	Прейскурант_id	id_price_list	INT	-	FK	Price-list	NOT NULL

Таблица 10. Посетитель

Посетитель - Visitor							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	Посетитель_id	id_visitor	INT	-	PK	-	NOT NULL
2	Фамилия_посетителя	visitor_surname	VARCHAR(40)	-	-	-	NOT NULL
3	Имя_посетителя	visitor_name	VARCHAR(15)	-	-	-	NOT NULL
4	Отчество_посетителя	visitor_patronymic	VARCHAR(20)	NULL	-	-	NULL
5	Дата рождения	visitor_date_of_birth	DATE	-	-	-	NOT NULL
6	Ответственный_id	id_responsible	INT	NULL	FK	Responsible	NULL
7	Наличие медицинской справки	medical_certificate	BOOLEAN	FALSE	-	-	NOT NULL
8	Телефон	visitor_phone	VARCHAR(15)	-	-	-	NOT NULL

Таблица 11. Абонемент

Абонемент - Pass							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	Абонемент_id	id_pass	INT	-	PK	-	NOT NULL
2	Тип_абонемента_id	id_type_of_pass	INT	-	FK	Type of pass	NOT NULL
3	Дата_начала	pass_start_date	DATE	-	-	-	NOT NULL
4	Дата_окончания	pass_finish_date	DATE	-	-	-	NOT NULL
5	Посетитель_id	id_visitor	INT	-	FK	Visitor	NOT NULL
6	Количество посещений	pass_number_visits	INT	2	-	-	NOT NULL

Таблица 12. Прейскурант

Прейскурант - Price-list							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	Прейскурант_id	id_price_list	INT	-	PK	-	NOT NULL
2	Стоимость	cost	DECIMAL(10, 2)	0.00	-	-	NOT NULL
3	Уровень_нагрузки_id	id_skill	INT	-	FK	Skilltest	NOT NULL
4	Объем_занятий	lesson_volume	INT	1	-	-	NOT NULL
5	Тип_абонемента_id	id_type_of_pass	INT	-	FK	Type of pass	NOT NULL

Таблица 14. Посещение

Посещение - Visit							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	Посещение_id	id_visit	INT	-	PK	-	NOT NULL
2	Дата_посещения	visit_date	DATE	-	-	-	NOT NULL
3	День_недели_id	id_weekday	INT	-	FK	Weekday	NOT NULL
4	Расписание_id	id_schedule	INT	-	FK	Schedule	NOT NULL
5	Администратор_id	id_admin	INT	-	FK	Admin	NOT NULL
6	Абонемент_id	id_pass	INT	-	FK	Pass	NOT NULL

3.2 Лингвистическое описание схемы

Таблицы:

1. Тип дня (день_id(int), тип дня(varchar(8))).
2. День недели (день_недели_id(int), день недели(varchar(11))).
3. Уровень нагрузки (уровень_нагрузки_id(int), сложность(varchar(7))).
4. Категория абонемента (категория_id(int), возрастной тип(varchar(8))).
5. Ответственный (ответственный_id(int), фамилия ответственного (varchar(40)), имя ответственного(varchar(15)), отчество ответственного(varchar(20)), телефон ответственного(varchar(15))).
6. Администратор (администратор_id(int), фамилия администратора (varchar(40)), имя администратора(varchar(15)), отчество администратора(varchar(20)), опыт работы(int)).
7. Вид абонемента (вид_абонемента_id(int), название вида абонемента(varchar(17)), категория абонемента(категория_id(int))).
8. Сетка расписания (расписание_id(int), тип дня(день_id(int)), начало сеанса (time), конец сеанса(time)).
9. Тип абонемента (тип_абонемента_id(int), сетка расписания (расписание_id(int)), дни сеанса(varchar(30)), вид абонемента (вид_абонемента_id(int))).
10. Посетитель (посетитель_id(int), фамилия посетителя(varchar(40)), имя посетителя(varchar(15)), отчество посетителя(varchar(20)), дата рождения(date), ответственный(ответственный_id(int)), наличие медицинской справки, телефон (varchar(15))).
11. Абонемент (абонемент_id(int), тип абонемента(тип_абонемента_id(int)), дата начала(date), дата окончания(date), посетитель(посетитель_id(int)), количество посещений(int)).
12. Прейскурант (прейскурант_id(int), стоимость(decimal(10,2)), уровень нагрузки(уровень_нагрузки_id(int)), объем занятий(int), тип абонемента(тип_абонемента_id(int))).

13. Покупка (покупка_id(int), дата покупки(date), администратор (администратор_id(int)), абонемент(абонемент_id(int)), посетитель (посетитель_id(int)), прейскурант(прейскурант_id(int))).
14. Посещение (посещение_id(int), дата посещения(date), день недели(день_недели_id(int)), сетка расписания(расписание_id(int)), администратор(администратор_id(int)), абонемент(абонемент_id(int))).

Таблица 15. Количество записей в таблицах

Количество записей в таблицах		
№	Название таблицы	Количество записей
1	Тип дня	2
2	День недели	7
3	Уровень нагрузки	3
4	Категория абонемента	2
5	Ответственный	5000
6	Администратор	50
7	Вид абонемента	5
8	Сетка расписания	14
9	Тип абонемента	200
10	Посетитель	10400
11	Абонемент	100000
12	Прейскурант	392
13	Покупка	51888
14	Посещение	250275

3.3 Графическое описание схемы базы данных на русском языке

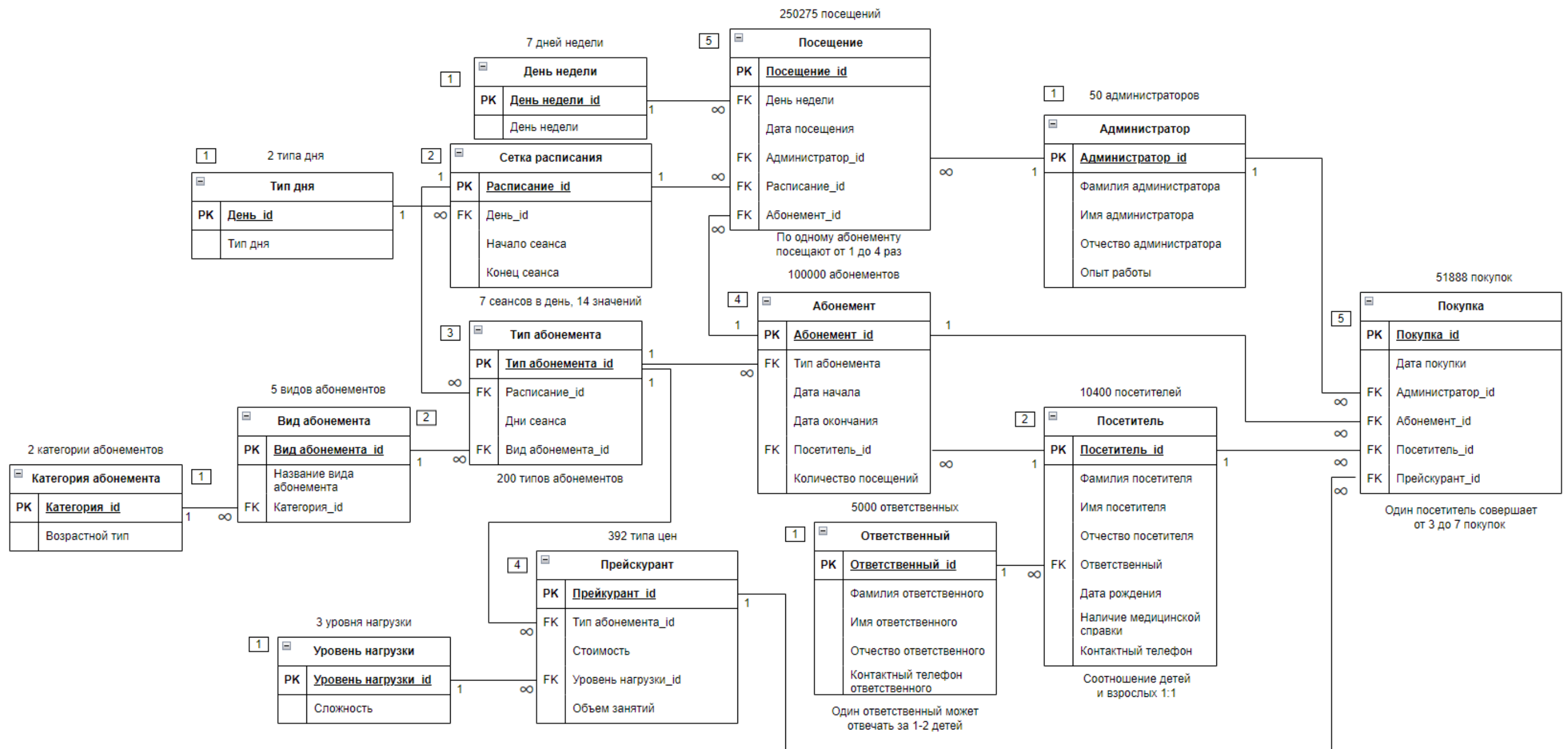


Рис. 3: Графическое описание схемы базы данных на русском языке

3.4 Графическое описание схемы базы данных на английском языке

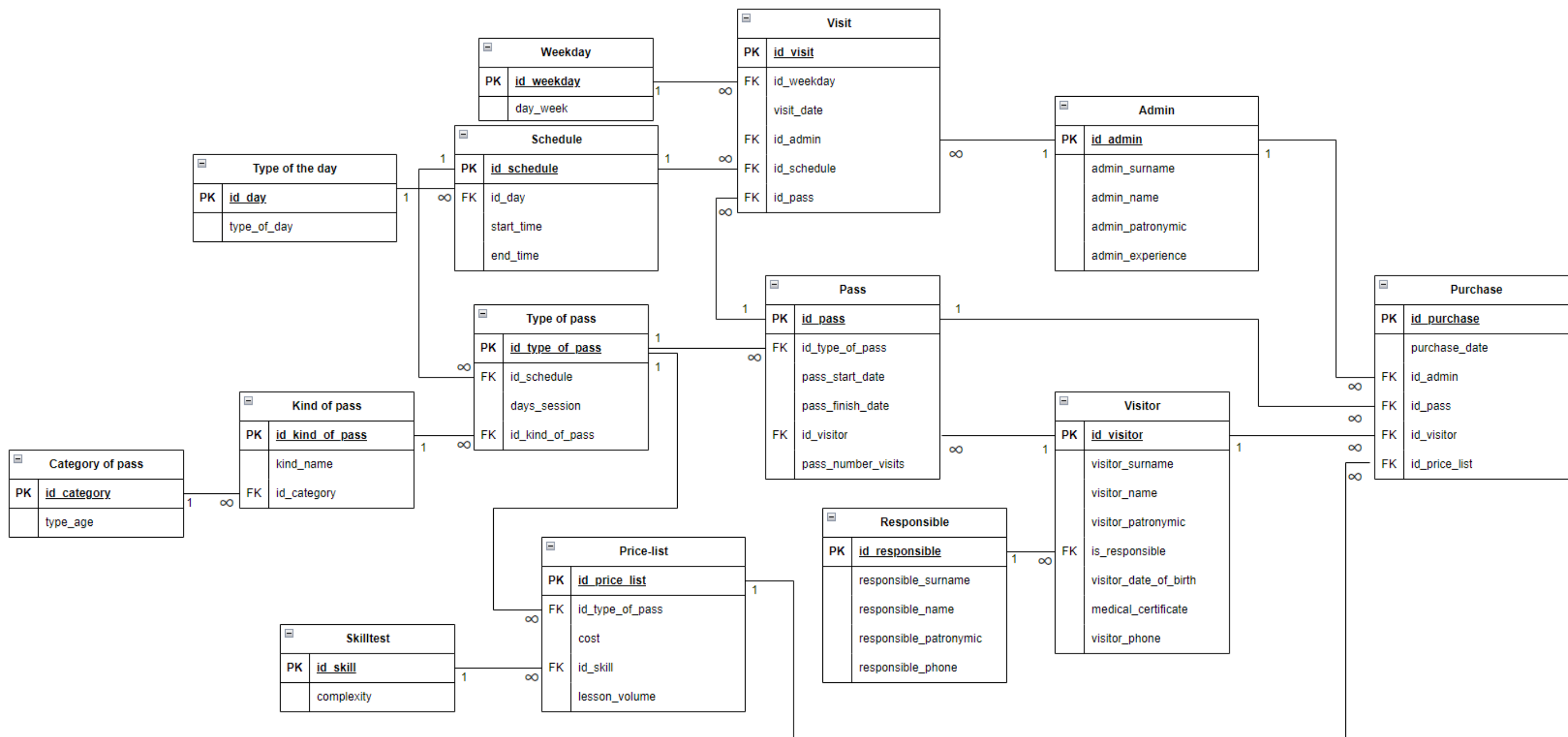


Рис. 4: Графическое описание схемы базы данных на английском языке

3.5 Создание базы данных и заполнение значениями

База данных и таблицы, содержащиеся в ней, были созданы с помощью SQL-запросов и командной строки MySQL [5]. Для обеспечения оптимальной производительности системы были созданы соответствующие индексы внешних ключей. Код создания базы данных представлен в приложении А. Графическое описание базы данных на русском и английском языках представлены на рисунках 3, 4.

База данных была заполнена случайными значениями, сгенерированными программой на языке Python 3. Для подключения к базе данных с использованием библиотеки pymysql [4] применялась настройка API, а именно были указаны хост, имя пользователя, пароль, имя базы данных и кодировка. С использованием настроенного подключения значения автоматически передавались в базу данных. Таблицы, содержащие слова-ри, были заполнены заранее вручную через стандартные SQL-команды INSERT [6]. Код заполнения таблиц базы данных представлен в приложении Б. Всего было выполнено 14 запросов для заполнения 14 таблиц, которые содержат 418238 значений.

4 Программирование

Лист проверки выполнения запросов
курсовой работы по дисциплине
«Теоретические основы баз данных» 4 семестр

Имя, отчество, фамилия Мартыченко Анна Павловна Группа 5130201/3000 3

Тема работы Функционирование работы бассейна на коммерческой основе

ТО		№ п.п.	поправить время: изменить параметры текст запроса	принято		
				запрос	отчет	
→	+	+	1	Найти всех посетителей, к-рые купили ^{разр} ад. типа "А" и продали адм Б, посещая в день недели С	☒	☒
	-	+	*	не менее 3 раз посещая	☒	☒
	-	+	2	count pass для взрослых со <u>слотами</u> уровнем нагрузки <u>интер-полюк</u>	☒	☒
	-	+	3	count посещений, к-рые зарег. каждый администратор	☒	☒
	-	+		история распределения + число покупок ↑	☒	☒
→	+	+	4	count посетителей с одинак. числом абонентов	☒	☒
				история распределения	☒	☒
	-	-	5	найти тип абонента, у к-рых count pass > чем у типа абонента А	☒	☒
				А	☒	☒
→	+	-	6	найти посетителей, к-рые ш раз не посещали бассейн в день А	☒	☒
				А/В +	☒	☒
	-	-	7	для 4 вида абонента admin и weekday count посещений	☒	☒
				3-d история	☒	☒
→	+	+	8	найти посетителей с max числом посещений	☒	☒
	-	+		min	☒	☒
				add несколько min/max и посмотреть +	☒	☒
			9		☐	☐
			10		☐	☐

4.1 Запрос №1.1

Необходимо найти всех посетителей, которые купили абонемент типа А у администратора Б и посещали в день недели С.

Текстовое объяснение: с помощью inner join соединяются таблицы Pass, Type_of_pass, Purchase, Admin, Visit и Weekday. Таблицы обрабатываются по PRIMARY KEY или с помощью id. Далее с помощью where выбираются только те строки, которые соответствуют указанным значениям type_of_pass.id_type_of_pass, admin.id_admin, weekday.id_weekday, эти значения находятся по индексу за константное время. Для того, чтобы строки не дублировались, используется группировка group by на visitor.id_visitor, type_of_pass.id_type_of_pass, admin.id_admin, visit.id_weekday. Затем строки сортируются по visitor.id_visitor. Графическое и табличное представление последовательности выполнения отчета представлено на рисунках 4 и 5.

Реляционная формула:

$$\pi \left(\begin{array}{l} \beta(\text{visitor.visitor_surname as «фамилия»}), \\ \beta(\text{visitor.visitor_name as «имя»}), \\ \beta(\text{visitor.visitor_patronymic as «отчество»}), \\ \beta(\text{type_of_pass.id_type_of_pass as «тип»}) \\ \beta(\text{admin.id_admin as «администратор»}) \\ \beta(\text{visit.id_weekday as «день недели»}) \end{array} \right)$$

$$\sigma(\text{type_of_pass.id_type_of_pass} = 149 \text{ and } \text{admin.id_admin} = 4 \\ \text{and } \text{weekday.id_weekday} = 1)$$

$$\begin{array}{l} \text{Type_of_pass} \bowtie_{(\text{pass.id_type_of_pass} = \text{type_of_pass.id_type_of_pass})} \text{Pass} \\ \text{Admin} \bowtie_{(\text{purchase.id_admin} = \text{admin.id_admin})} \text{Purchase} \\ \text{Weekday} \bowtie_{(\text{visit.id_weekday} = \text{weekday.id_weekday})} \text{Visit} \\ \text{Visitor} \bowtie_{(\text{visitor.id_visitor} = \text{pass.id_visitor})} \text{Pass} \\ \gamma(\text{visitor.id_visitor}, \text{type_of_pass.id_type_of_pass}, \\ \text{admin.id_admin}, \text{visit.id_weekday}) \\ \tau(\text{visitor.id_visitor}) \end{array}$$

Листинг 1. Запрос 1.1

```
1 SELECT
2 visitor.visitor_surname as "фамилия",
3 visitor.visitor_name as "имя",
4 visitor.visitor_patronymic as "отчество",
```

```

5 type_of_pass.id_type_of_pass as "тип",
6 admin.id_admin as "администратор",
7 visit.id_weekday as "день недели"
8 FROM Visitor
9 INNER JOIN Pass ON visitor.id_visitor = pass.id_visitor
10 INNER JOIN Type_of_pass ON pass.id_type_of_pass = type_of_pass.
   id_type_of_pass
11 INNER JOIN Purchase ON pass.id_pass = purchase.id_pass
12 INNER JOIN Admin ON purchase.id_admin = admin.id_admin
13 INNER JOIN Visit ON pass.id_pass = visit.id_pass
14 INNER JOIN Weekday ON visit.id_weekday = weekday.id_weekday
15 WHERE
16 type_of_pass.id_type_of_pass=149
17 AND admin.id_admin = 4
18 AND weekday.id_weekday = 1
19 GROUP BY visitor.id_visitor, type_of_pass.id_type_of_pass, admin.
   id_admin, visit.id_weekday
20 ORDER BY visitor.id_visitor;

```

В результирующей таблице было выведено 3 строки, время выполнения запроса составляет 0.016 секунд.

Фамилия	Имя	Отчество	Тип	Администратор	День недели
Ларин	Адам	Климович	149	4	понедельник
Семенов	Арина	Матвейовна	149	4	понедельник
Хомяченко	Артур	Альбертович	149	4	понедельник

Рис. 3. Результат запроса 1.1

id	select_type	table	type	possible_keys	key	key...	ref	rows	filtered	Extra
1	SIMPLE	Type_of_pass	const	PRIMARY	PRIMARY	4	const	1	100.00	Using index; Using temporary; Using...
1	SIMPLE	Admin	const	PRIMARY	PRIMARY	4	const	1	100.00	Using index
1	SIMPLE	Weekday	const	PRIMARY	PRIMARY	4	const	1	100.00	
1	SIMPLE	Pass	ref	PRIMARY, id_type_of_pass, id_visitor	id_type_of_pass	4	const	552	100.00	
1	SIMPLE	Purchase	ref	id_admin, id_pass	id_pass	4	swimming_pool.Pass.id_pass	1	3.85	Using where
1	SIMPLE	Visit	ref	id_weekday, id_pass	id_pass	4	swimming_pool.Pass.id_pass	2	28.57	Using where
1	SIMPLE	Visitor	eq_ref	PRIMARY	PRIMARY	4	swimming_pool.Pass.id_visitor	1	100.00	

Рис. 4. Табличное представление последовательности выполнения запроса

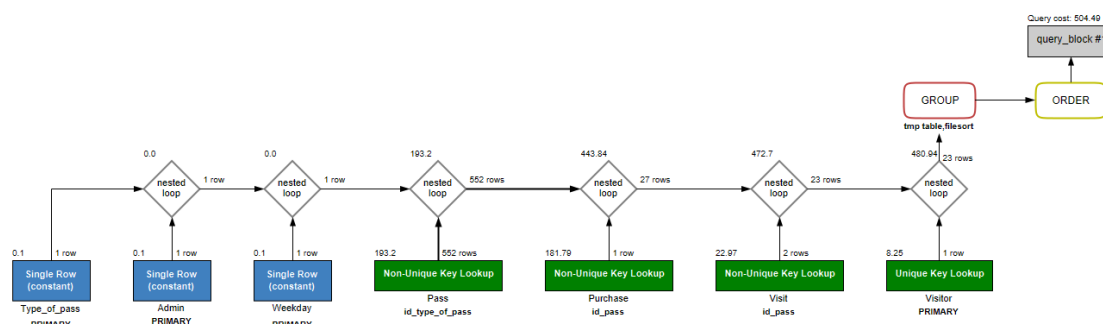


Рис. 5. Графическое представление последовательности выполнения запроса

4.2 Запрос №1.2

Необходимо найти всех посетителей, которые купили абонемент типа А у администратора Б и посещали в день недели С не менее трех раз.

Графическое и табличное представление последовательности выполнения отчета представлено на рисунках 7 и 8.

Реляционная формула:

$$\pi \left(\begin{array}{l} \beta(\text{visitor.visitor_surname as «фамилия»}), \\ \beta(\text{visitor.visitor_name as «имя»}), \\ \beta(\text{visitor.visitor_patronymic as «отчество»}), \\ \beta(\text{type_of_pass.id_type_of_pass as «тип»}) \\ \beta(\text{admin.id_admin as «администратор»}) \\ \beta(\text{visit.id_weekday as «день недели»}) \\ \beta(\text{count(visit.id_visit) as «количество посещений»}) \end{array} \right)$$

$$\sigma(\text{type_of_pass.id_type_of_pass} = 149 \text{ and } \text{admin.id_admin} = 4 \\ \text{and } \text{weekday.id_weekday} = 1 \text{ and } \text{COUNT(visit.id_visit)} \geq 3)$$

$$\text{Type_of_pass} \bowtie_{(\text{pass.id_type_of_pass} = \text{type_of_pass.id_type_of_pass})} \text{Pass}$$

$$\text{Admin} \bowtie_{(\text{purchase.id_admin} = \text{admin.id_admin})} \text{Purchase}$$

$$\text{Weekday} \bowtie_{(\text{visit.id_weekday} = \text{weekday.id_weekday})} \text{Visit}$$

$$\text{Visitor} \bowtie_{(\text{visitor.id_visitor} = \text{pass.id_visitor})} \text{Pass}$$

$$\gamma(\text{visitor.visitor_surname, visitor.visitor_name, visitor.visitor_patronymic} \\ \text{type_of_pass.id_type_of_pass, admin.id_admin,} \\ \text{visit.id_weekday, COUNT(visit.id_visit) })$$

$$\tau(\text{visitor.visitor_surname, visitor.visitor_name, visitor.visitor_patronymic})$$

Листинг 2. Запрос 1.2

```

1  SELECT
2  visitor.visitor_surname as "фамилия",
3  visitor.visitor_name as "имя",
4  visitor.visitor_patronymic as "отчество",
5  type_of_pass.id_type_of_pass as "тип",
6  admin.id_admin as "администратор",
7  visit.id_weekday as "день недели"
8  COUNT(visit.id_visit) AS "количество посещений"
9  FROM Visitor
10 INNER JOIN Pass ON visitor.id_visitor = pass.id_visitor
11 INNER JOIN Type_of_pass ON pass.id_type_of_pass = type_of_pass.
   id_type_of_pass
12 INNER JOIN Purchase ON pass.id_pass = purchase.id_pass

```

```

13 INNER JOIN Admin ON purchase.id_admin = admin.id_admin
14 INNER JOIN Visit ON pass.id_pass = visit.id_pass
15 INNER JOIN Weekday ON visit.id_weekday = weekday.id_weekday
16 WHERE
17 type_of_pass.id_type_of_pass=149
18 AND admin.id_admin = 4
19 AND weekday.id_weekday = 1
20 GROUP BY
21 visitor.visitor_surname, visitor.visitor_name, visitor.
    visitor_patronymic, type_of_pass.id_type_of_pass, admin.id_admin,
    visit.id_weekday
22 HAVING
23 COUNT(visit.id_visit) >= 3
24 ORDER BY visitor.visitor_surname, visitor.visitor_name, visitor.
    visitor_patronymic;

```

В результирующей таблице было выведено 2 строки, время выполнения запроса составляет 0.031 секунды.

Фамилия	Имя	Отчество	Тип	Администратор	День недели	Количество посещений
Ларин	Адам	Климович	149	4	понедельник	3
Семенов	Арина	Матвейовна	149	4	понедельник	3

Рис. 6. Результат запроса 1.2

id	select_type	table	type	possible_keys	key	key...	ref	rows	filtered	Extra
1	SIMPLE	Type_of_pass	const	PRIMARY	PRIMARY	4	const	1	100.00	Using index; Using temporary; Using...
1	SIMPLE	Admin	const	PRIMARY	PRIMARY	4	const	1	100.00	Using index
1	SIMPLE	Weekday	const	PRIMARY	PRIMARY	4	const	1	100.00	
1	SIMPLE	Pass	ref	PRIMARY; id_type_of_pass; id_visitor	id_type_of_pass	4	const	552	100.00	
1	SIMPLE	Purchase	ref	id_admin; id_pass	id_pass	4	swimming_pool.Pass.id_pass	1	3.85	Using where
1	SIMPLE	Visit	ref	id_weekday; id_pass	id_pass	4	swimming_pool.Pass.id_pass	2	28.57	Using where
1	SIMPLE	Visitor	eq_ref	PRIMARY	PRIMARY	4	swimming_pool.Pass.id_visitor	1	100.00	

Рис. 7. Табличное представление последовательности выполнения запроса

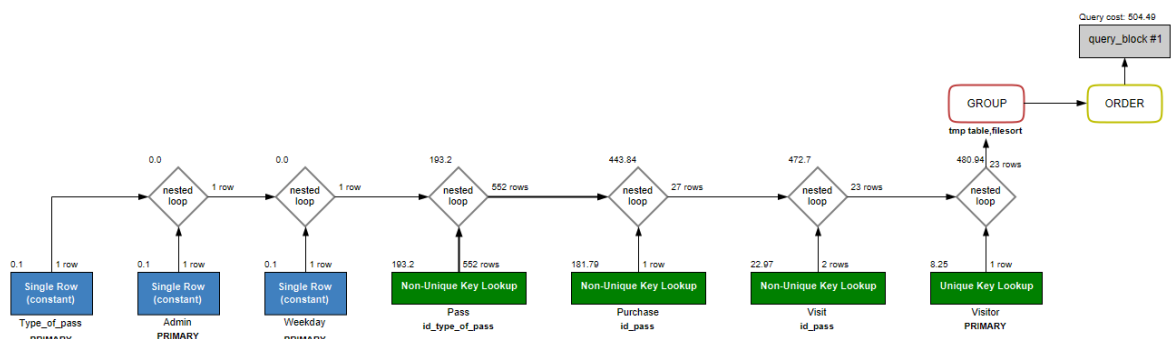


Рис. 8. Графическое представление последовательности выполнения запроса

4.3 Запрос №2

Необходимо посчитать количество абонементов с уровнем нагрузки А для взрослых.

Графическое и табличное представление последовательности выполнения отчета представлено на рисунках 10 и 11.

Реляционная формула:

$$\pi \left(\begin{array}{l} \beta(\text{skilltest.complexity as «тип нагрузки»}), \\ \beta(\text{category_of_pass.type_age as «возраст»}), \\ \beta(\text{COUNT(distinct pass.id_pass) as «количество абонементов»}), \end{array} \right) \\ \sigma(\text{skilltest.complexity = «сложный» and category_of_pass.type_age = «взрослый»}) \\ \text{Category_of_pass} \bowtie_{(\text{kind_of_pass.id_category} = \text{category_of_pass.id_category})} \text{Kind_of_pass} \\ \text{Skilltest} \bowtie_{(\text{price_list.id_skill} = \text{skilltest.id_skill})} \text{Price_list} \\ \text{Type_of_pass} \bowtie_{(\text{pass.id_type_of_pass} = \text{type_of_pass.id_type_of_pass})} \text{Pass}$$

Листинг 3. Запрос 1.2

```

1 SELECT
2 skilltest.complexity as "тип нагрузки",
3 category_of_pass.type_age as "возраст",
4 COUNT(distinct pass.id_pass) as "количество абонементов"
5 FROM Pass
6 INNER JOIN Type_of_pass ON pass.id_type_of_pass = type_of_pass.
   id_type_of_pass
7 INNER JOIN Kind_of_pass ON type_of_pass.id_kind_of_pass = kind_of_pass.
   id_kind_of_pass
8 INNER JOIN Category_of_pass ON kind_of_pass.id_category =
   category_of_pass.id_category
9 INNER JOIN Price_list ON type_of_pass.id_type_of_pass = price_list.
   id_type_of_pass
10 INNER JOIN Skilltest ON price_list.id_skill = skilltest.id_skill
11 WHERE skilltest.complexity = "сложный" AND category_of_pass.type_age = "
   взрослый";

```

В результирующей таблице была выведена 1 строка, время выполнения запроса составляет 0.031 секунд.

	тип нагрузки	возраст	количество абонементов
►	сложный	взрослый	29412

Рис. 9. Результат запроса 2

id	select_type	table	type	possible_keys	key	key...	ref	rows	filtered	Extra
1	SIMPLE	Category_of_pass	ALL	PRIMARY				2	50.00	Using where
1	SIMPLE	Skilltest	ALL	PRIMARY				3	33.33	Using where; Using join buffer (hash...
1	SIMPLE	Kind_of_pass	ref	PRIMARY,id_category	id_category	4	swimming_pool.Category_of...	2	100.00	Using index
1	SIMPLE	Type_of_pass	ref	PRIMARY,id_kind_of_pass	id_kind_of_pass	4	swimming_pool.Kind_of_pas...	40	100.00	Using index
1	SIMPLE	Price_list	ref	id_skill,id_type_of_pass	id_type_of_pass	4	swimming_pool.Type_of_pas...	1	33.33	Using where
1	SIMPLE	Pass	ref	id_type_of_pass	id_type_of_pass	4	swimming_pool.Type_of_pas...	490	100.00	Using index

Рис. 10. Табличное представление последовательности выполнения запроса

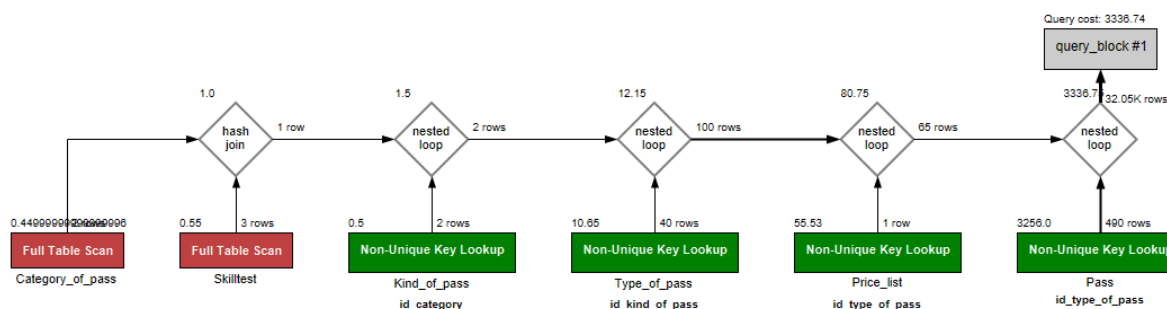


Рис. 11. Графическое представление последовательности выполнения запроса

4.4 Запрос №3.1

Необходимо посчитать количество посещений, которые зарегистрировал каждый администратор.

Графическое и табличное представление последовательности выполнения отчета представлено на рисунках 12 и 13.

Реляционная формула:

$$\pi \left(\begin{array}{l} \beta(\text{admin.admin_surname as «фамилия»}), \\ \beta(\text{admin.admin_name as «имя»}), \\ \beta(\text{admin.admin_patronymic as «отчество»}), \\ \beta(\text{COUNT(visit.id_visit) as «посещения»}), \end{array} \right) \\ \text{Admin} \bowtie_{(\text{admin.id_admin} = \text{visit.id_admin})} \text{Visit} \\ \gamma(\text{admin.admin_surname, admin.admin_name, admin.admin_patronymic})$$

Листинг 4. Запрос 3.1

```

1 SELECT
2 admin.admin_surname as "фамилия",
3 admin.admin_name as "имя",
4 admin.admin_patronymic as "отчество",
5 COUNT(visit.id_visit) as "посещения"
6 FROM Admin
7 INNER JOIN Visit ON admin.id_admin = visit.id_admin
8 GROUP BY admin.admin_surname, admin.admin_name, admin.admin_patronymic;
```

В результирующей таблице было выведено 50 строк, время выполнения запроса составляет 0.828 секунд.

id	select_type	table	type	possible_keys	key	key...	ref	rows	filtered	Extra
1	SIMPLE	Admin	ALL	PRIMARY				50	100.00	Using temporary
1	SIMPLE	Visit	ref	id_admin	id_admin	4	swimming_pool.Adminid_ad...	4902	100.00	Using index

Рис. 12. Табличное представление последовательности выполнения запроса

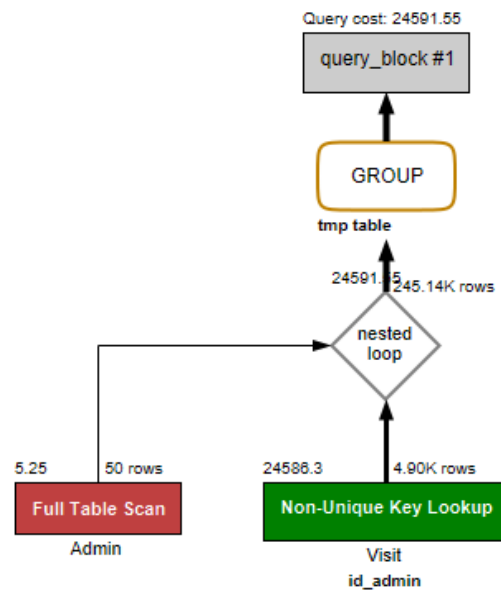


Рис. 13. Графическое представление последовательности выполнения запроса

фамилия	имя	отчество	посещения
Алешин	Меркурий	Марковна	5135
Андрианов	Валентина	Оскарович	5075
Архипов	Герман	Всеволодовна	5021
Безруков	Людмила	Арсенийовна	4951
Борисов	Август	Геннадийовна	5017
Голиков	Каролина	Семёновна	4996
Емельянов	Ангела	Захаровна	4908
Еременко	Клим	Владимировна	4941
Еремин	Евгений	Вениаминовна	5058
Жирин	Анастасия	Кириллович	5068
Журавленко	Галина	Сергейович	4919
Завьялов	Марина	Арсенийович	5143
Заец	Ксения	Брониславовна	5053
Заец	Любомир	Русланович	5013
Казаков	Евгений	Валерийовна	4892
Коновалов	Илларион	Меркурийовна	4902
Косарев	Жанна	Максимовна	5047
Котов	Нестор	Маркович	4922
Кузнецов	Снежана	Григорийовна	5068
Медведев	Геннадий	Родионовна	4919
Медведев	Клавдия	Аркадийовна	4930
Новиков	Август	Алексейович	5089
Павловский	Павел	Вячеславовна	4983
Панин	Василина	Романовна	5082
Рожков	Виктор	Павеловна	5095
Рублев	Всеволод	Алексейович	5013
Русаков	Елизавета	Гарриович	5096

Рис. 14. Фрагмент результата запроса 3.1

4.5 Запрос №3.2

Необходимо посчитать количество покупок, которые зарегистрировал каждый администратор.

Графическое и табличное представление последовательности выполнения отчета представлено на рисунках 15 и 16.

Реляционная формула:

$$\begin{aligned}
 & \pi \left(\begin{array}{l} \beta(\text{admin.admin_surname as «фамилия»}), \\ \beta(\text{admin.admin_name as «имя»}), \\ \beta(\text{admin.admin_patronymic as «отчество»}), \\ \beta(\text{COUNT(purchase.id_purchase) as «покупки»}), \end{array} \right) \\
 & \text{Admin} \bowtie_{(\text{admin.id_admin} = \text{purchase.id_admin})} \text{Purchase} \\
 & \gamma(\text{admin.admin_surname, admin.admin_name, admin.admin_patronymic})
 \end{aligned}$$

```

1 SELECT
2 admin.admin_surname as "фамилия",
3 admin.admin_name as "имя",
4 admin.admin_patronymic as "отчество",
5 COUNT(purchase.id_purchase) as "покупки"
6 FROM Admin
7 INNER JOIN Purchase ON admin.id_admin = purchase.id_admin
8 GROUP BY admin.admin_surname, admin.admin_name, admin.admin_patronymic;

```

В результирующей таблице было выведено 50 строк, время выполнения запроса составляет 0.140 секунд.

id	select_type	table	type	rows	filtered	Extra
1	SIMPLE	Admin	ALL	50	100.00	Using temporary
1	SIMPLE	Visit	ref	4902	100.00	Using index

Рис. 15. Табличное представление последовательности выполнения запроса

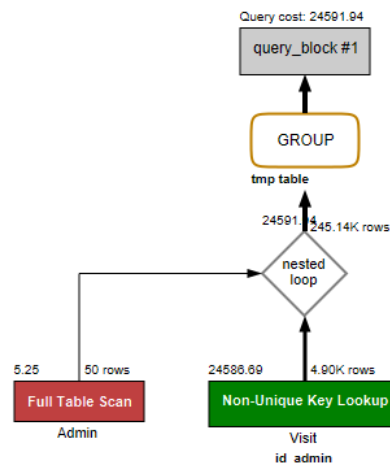


Рис. 16. Графическое представление последовательности выполнения запроса

фамилия ▲	имя	отчество	покупки
Алешин	Меркурий	Марковна	1072
Андрианов	Валентина	Оскарович	1035
Архипов	Герман	Всеволодовна	1037
Безруков	Людмила	Арсенийовна	1026
Борисов	Август	Геннадийовна	1012
Голиков	Каролина	Семёновна	1089
Емельянов	Ангела	Захаровна	983
Еременко	Клим	Владимировна	1018
Еремин	Евгений	Вениаминовна	1044
Жири	Анастасия	Кириллович	1085
Журавленко	Галина	Сергейович	1038
Завьялов	Марина	Арсенийович	1107
Заец	Любомир	Русланович	1036
Заец	Ксения	Брониславовна	1049

Рис. 17. Фрагмент результата запроса 3

4.6 Гистограмма распределения для запросов 3.1 и 3.2

На рисунке 18 представлена гистограмма распределения по администраторам количества посещений и количества покупок, зарегистрированных каждым администратором. Синим цветом обозначено количество посещений, красным - количество покупок.

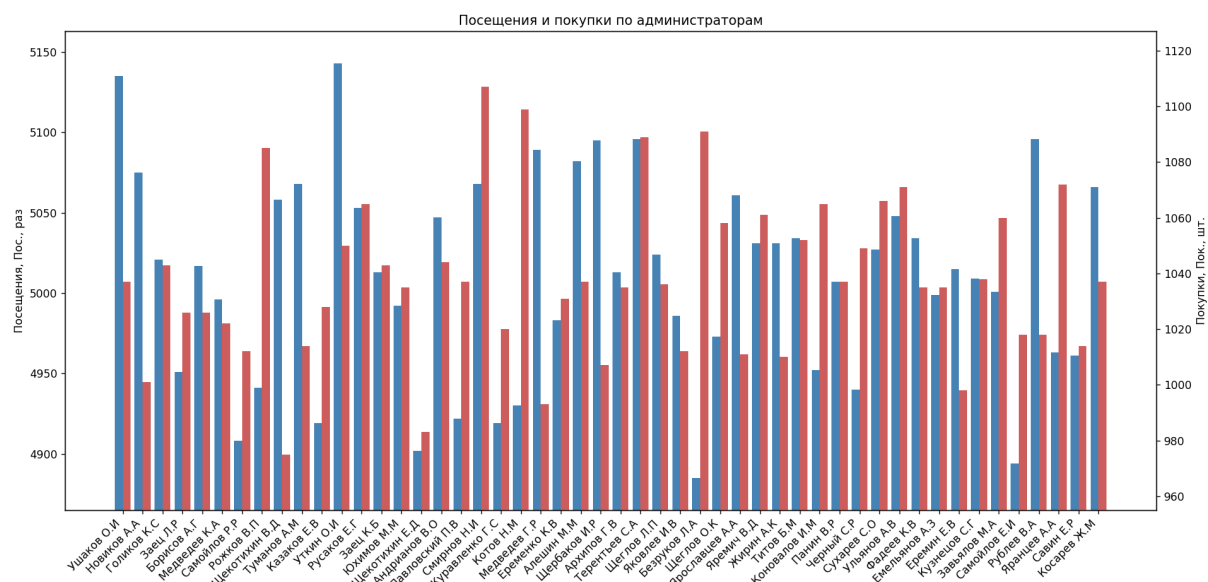


Рис. 18. Гистограмма распределения количества заявок и количества покупок, зарегистрированных каждым администратором

4.7 Запрос №4

Необходимо посчитать количество посетителей с одинаковым числом абонементов.

Текстовое объяснение: внутренний подзапрос группирует данные по посетителям (`visitor.id_visitor`) и подсчитывает количество связанных с каждым посетителем абонементов. Таблица `Visitor` обрабатывается по первичному ключу, таблица `Pass` соединяется с `Visitor` по `id_visitor`. Во внешнем запросе выполняется подсчет количества посетителей с одинаковым числом абонементов с помощью `count`, а затем данные группируются по `counter` - количеству абонементов, полученных во внутреннем подзапросе. Для удобства представления информации данные сортируются по `counter`. Графическое и табличное представление последовательности выполнения отчета представлено на рисунках 20 и 21.

Реляционная формула:

$$\pi \left(\begin{array}{l} \beta(\text{counter as «число абонементов»}), \\ \beta(\text{COUNT(id_visitor) as «число посетителей»}) \end{array} \right) \\ \gamma(\text{counter}) \\ \tau(\text{counter}) \\ \text{где counter} \\ \pi \left(\begin{array}{l} \beta(\text{visitor.id_visitor, }) \\ \beta(\text{COUNT(pass.id_pass) as counter}) \end{array} \right) \\ \text{Visitor} \bowtie_{(\text{visitor.id_visitor} = \text{pass.id_visitor})} \text{Pass} \\ \gamma(\text{visitor.id_visitor})$$

Листинг 6. Запрос 4

```
1 SELECT counter as "число абонементов",
2 COUNT(id_visitor) as "число посетителей"
3 FROM(
4 SELECT
5 visitor.id_visitor,
6 COUNT(pass.id_pass) as counter
7 FROM Visitor
8 JOIN Pass ON visitor.id_visitor = pass.id_visitor
9 GROUP BY visitor.id_visitor
10 ) as visitor_pass_count
11 GROUP BY counter
12 ORDER BY counter;
```

В результирующей таблице было выведено 23 строки, время выполнения запроса составляет 0.047 секунд.

Гистограмма распределения количества посетителей с одинаковым числом абонементов представлена на рисунке 22.

id	select_type	table	type	possible_keys	key	key...	ref	rows	filtered	Extra
1	PRIMARY	<derived2>	ALL					98940	100.00	Using temporary; Using filesort
2	DERIVED	Visitor	index	PRIMARY,id_resp...	PRIMARY	4		10391	100.00	Using index
2	DERIVED	Pass	ref	id_visitor	id_visitor	4	swimming_pool.Visitorid_vis...	9	100.00	Using index

Рис. 19. Табличное представление последовательности выполнения запроса

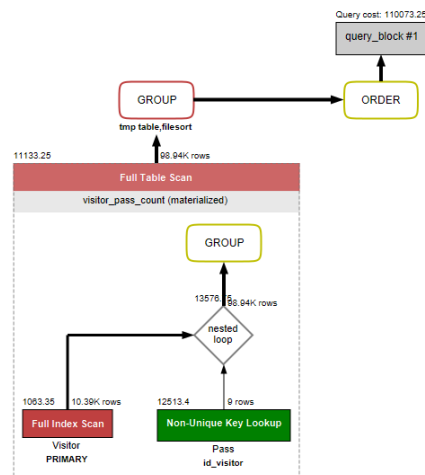


Рис. 20. Графическое представление последовательности выполнения запроса

число абонементов	число посетителей
1	7
2	31
3	108
4	246
5	509
6	721
7	1047
8	1285
9	1304
10	1303
11	1141
12	916
13	619
14	467
15	310
16	181
17	77
18	64
19	34
20	18
21	8
22	2
24	1

Рис. 21. Фрагмент результата запроса 4

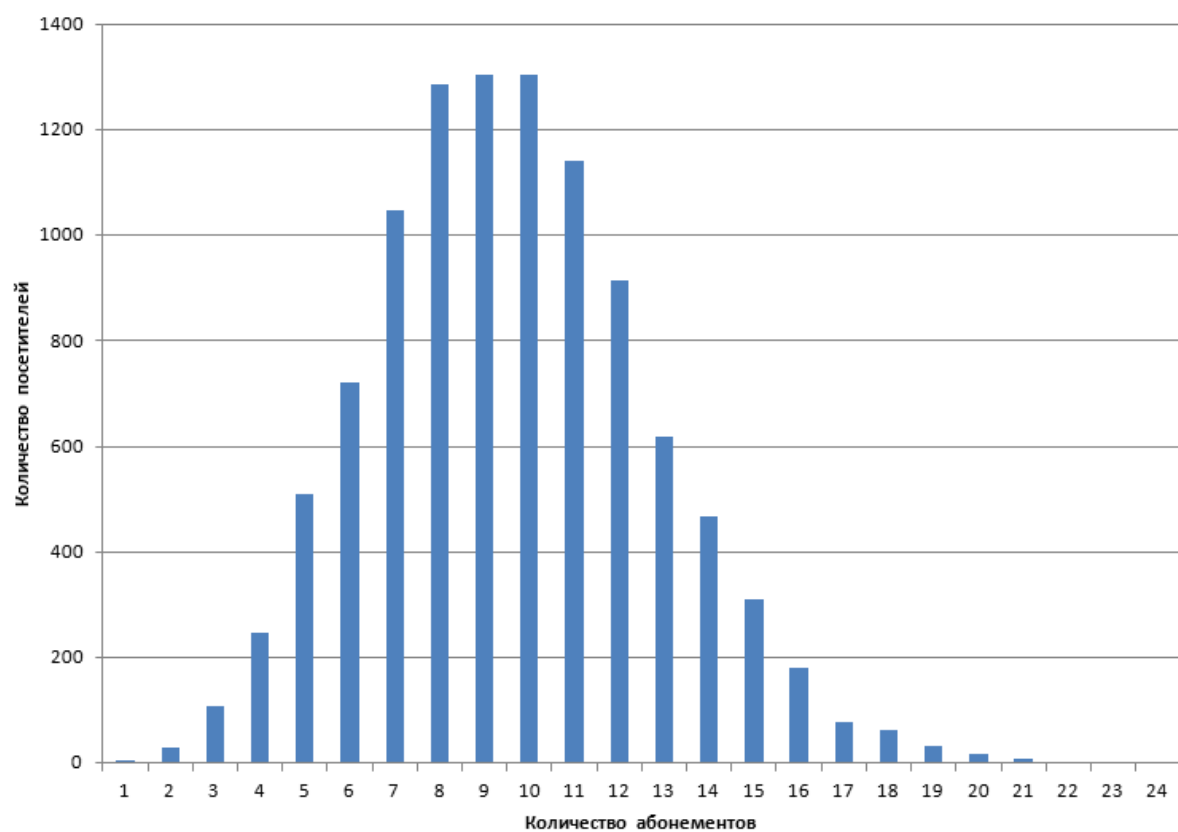


Рис. 22. Гистограмма распределения количества посетителей с одинаковым числом абонементов

4.8 Запрос №5

Необходимо найти типы абонементов, у которых число абонементов больше, чем у типа абонемента А.

Графическое и табличное представление последовательности выполнения отчета представлено на рисунках 23 и 24.

Листинг 7. Запрос 5

```

1 SELECT
2 type_of_pass.id_type_of_pass as "ID типа абонемента",
3 COUNT(pass.id_pass) as "количество абонементов"
4 FROM
5 Type_of_pass
6 INNER JOIN Pass ON type_of_pass.id_type_of_pass = pass.id_type_of_pass
7 GROUP BY type_of_pass.id_type_of_pass
8 HAVING COUNT(pass.id_pass) > (
9 SELECT COUNT(p.id_pass)
10 FROM Type_of_pass as top
11 INNER JOIN Pass as p ON top.id_type_of_pass = p.id_type_of_pass
12 WHERE top.id_type_of_pass = 38
13 GROUP BY top.id_type_of_pass
14 )
15 ORDER BY COUNT(pass.id_pass) asc;

```

В результирующей таблице было выведено 119 строк, время выполнения запроса составляет 0.078 секунд.

id	select_type	table	type	possible_keys	key	key...	ref	rows	filtered	Extra
1	PRIMARY	Type_of_pass	index	PRIMARY,id_sche...	PRIMARY	4		200	100.00	Using index; Using temporary; Using...
1	PRIMARY	Pass	ref	id_type_of_pass	id_type_of_pass	4	swimming_pool.Type_of_pas...	490	100.00	Using index
2	SUBQUERY	top	const	PRIMARY	PRIMARY	4	const	1	100.00	Using index
2	SUBQUERY	p	ref	id_type_of_pass	id_type_of_pass	4	const	494	100.00	Using index

Рис. 23. Табличное представление последовательности выполнения запроса

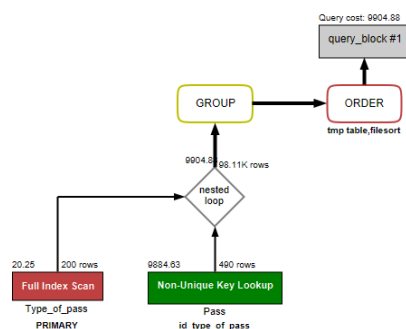


Рис. 24. Графическое представление последовательности выполнения запроса

ID типа абонента	количество абонентов
34	495
129	495
159	495
167	495
198	495
10	496
31	496
168	496
26	497
54	497
98	497
134	497
152	497
162	497
175	497

Рис. 25. Фрагмент результата запроса 5

4.9 Запрос №6

Необходимо найти всех посетителей, которые ни разу не посещали бассейн в день недели А.

Текстовое описание: внутренний подзапрос ищет абонементы для текущего посетителя и посещения для найденных абонементов, обе таблицы получают доступ по ссылке. Также в подзапросе фильтруются записи по visit.id_weekday. Внешний запрос сканирует всю таблицу Visitor, для каждой строки выполняя подзапрос, отбирая только те записи, которые соответствуют условию. После этого результаты сортируются по ФИО посетителя для удобства представления.

Графическое и табличное представление последовательности выполнения отчета представлено на рисунках 23 и 24.

Листинг 8. Запрос 6

```

1 SELECT
2 visitor.visitor_surname as "фамилия",
3 visitor.visitor_name as "имя",
4 visitor.visitor_patronymic as "отчество"
5 FROM Visitor
6 WHERE NOT EXISTS (
7   SELECT 1
8   FROM Pass
9   JOIN Visit ON pass.id_pass = visit.id_pass
10  WHERE pass.id_visitor = visitor.id_visitor
11  AND visit.id_weekday = 3
12 )

```

```
13 ORDER BY visitor.visitor_surname, visitor.visitor_name, visitor.
    visitor_patronymic;
```

В результирующей таблице было выведено 527 строк, время выполнения запроса составляет 0.125 секунд.

id	select_type	table	type	possible_keys	key	key...	ref	rows	filtered	Extra
1	SIMPLE	Visitor	ALL					10391	100.00	Using filesort
1	SIMPLE	<subquery2>	eq_ref	<auto_distinct_ke...	<auto_distinct_ke...	5	swimming_pool.Visitorid_vis...	1	100.00	Using where; Not exists
2	MATERIALIZED	Visit	ref	id_weekday,id_pa...	id_weekday	4	const	61536	100.00	
2	MATERIALIZED	Pass	eq_ref	PRIMARY,id_visitor	PRIMARY	4	swimming_pool.Visit.id_pass	1	100.00	

Рис. 26. Табличное представление последовательности выполнения запроса

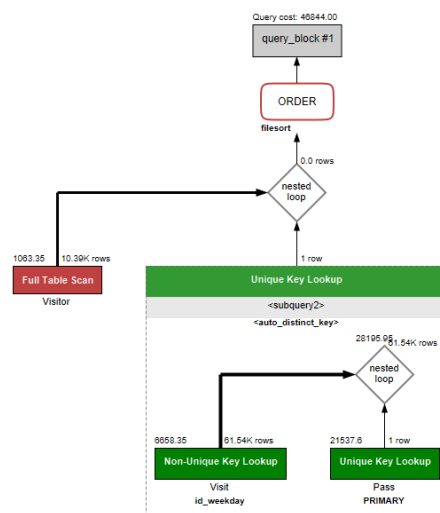


Рис. 27. Графическое представление последовательности выполнения запроса

Абрамов	Ксения	Демьяновна
Абрамов	Любовь	Изяславовна
Абрамов	Мелания	Милановна
Авдеев	Богдан	Иванович
Авдеев	Назар	Изяславович
Агапов	Вероника	Савелийович
Агапов	Наталия	NULL
Александров	Константин	Аркадийович
Александров	Павел	Николайовна
Александров	Регина	Еремейовна
Александров	Роза	Несторовна
Алехин	Капитолина	Григорийовна
Алехин	Нелли	Савелийович
Алешин	Григорий	Константин...

Рис. 28. Фрагмент результата запроса 6

4.10 Запрос №7

Необходимо для каждого администратора и дня недели посчитать количество посещений.

Графическое и табличное представление последовательности выполнения отчета представлено на рисунках 29 и 30.

Листинг 9. Запрос 7

```
1 SELECT
2 visit.id_admin AS "администратор",
3 visit.id_weekday AS "день недели",
4 COUNT(visit.id_visit) AS "количество посещений"
5 FROM Visit
6 GROUP BY visit.id_admin, visit.id_weekday
7 ORDER BY visit.id_admin, visit.id_weekday;
```

В результирующей таблице было выведено 350 строк, время выполнения запроса составляет 0.156 секунд.

Гистограмма распределения количества посетителей с одинаковым числом абонементов представлена на рисунке 32.

id	select_type	table	type	rows	filtered	Extra
1	SIMPLE	Visit	ALL	250043	100.00	Using temporary; Using filesort

Рис. 29. Табличное представление последовательности выполнения запроса

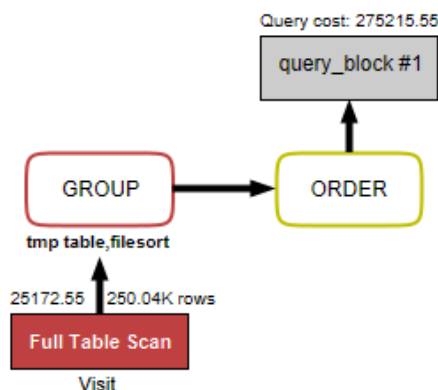


Рис. 30. Графическое представление последовательности выполнения запроса

администратор	день недели	количество посещений
1	1	716
1	2	714
1	3	721
1	4	726
1	5	721
1	6	708
1	7	725
2	1	766
2	2	688
2	3	690
2	4	747
2	5	762
2	6	724

Рис. 31. Фрагмент результата запроса 7

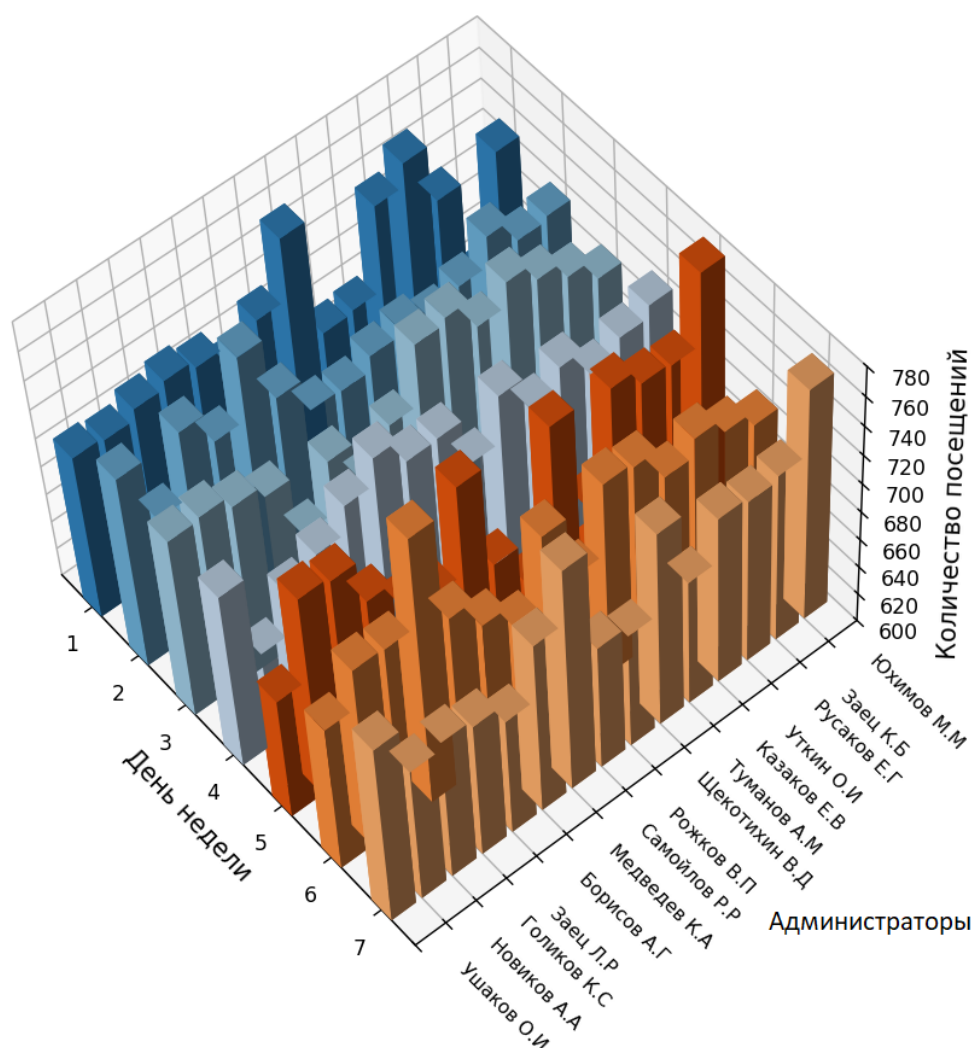


Рис. 32. Гистограмма запроса

4.11 Запрос №8.1

Необходимо найти посетителей с максимальным числом посещений.

Текстовое объяснение: в подзапросе таблица «Visit» соединяется с таблицей «Pass» для подсчета посещений каждого посетителя. Результат группируется по pass.id_visitor и сортируется по убыванию, затем выбирается одно значение, которое является максимальным. В запросе таблицу «Visitor» соединяется с таблицами «Pass» и «Visit», происходит группировка по каждому посетителю visitor.id_visitor. После этого строки фильтруются по условию, заданному в подзапросе. Графическое и табличное представление последовательности выполнения отчета представлено на рисунках 33 и 34.

Реляционная формула:

$$\pi \left(\begin{array}{l} \beta(\text{visitor.visitor_surname as «фамилия»}), \\ \beta(\text{visitor.visitor_name as «имя»}), \\ \beta(\text{visitor.visitor_patronymic as «отчество»}), \\ \beta(\text{count(visit.id_visit) as «количество посещений»}) \end{array} \right) \\ \sigma(\text{количество посещений} = \text{max_visits}) \\ \text{Visitor} \bowtie_{(\text{visitor.id_pass} = \text{pass.id_pass})} \\ \text{Pass} \bowtie_{(\text{pass.id_visitor} = \text{visit.id_visitor})} \text{Visit} \\ \gamma(\text{id_visitor})$$

Листинг 10. Запрос 8.1

```
1 SELECT
2 visitor.visitor_surname as "фамилия",
3 visitor.visitor_name as "имя",
4 visitor.visitor_patronymic as "отчество",
5 COUNT(visit.id_visit) as "количество посещений"
6 FROM Visit
7 INNER JOIN Pass ON visit.id_pass = pass.id_pass
8 INNER JOIN Visitor ON pass.id_visitor = visitor.id_visitor
9 GROUP BY visitor.id_visitor
10 HAVING COUNT(visit.id_visit) = (
11 SELECT COUNT(visit.id_visit) as count_visit
12 FROM Visit
13 INNER JOIN Pass ON visit.id_pass = pass.id_pass
14 GROUP BY pass.id_visitor
15 ORDER BY count_visit DESC
16 LIMIT 1
17 );
```

В результирующей таблице было выведено 2 строки, время выполнения запроса составляет 0.625 секунд.

id	select_type	table	type	rows	filtered	Extra
1	PRIMARY	Visitor	index	10391	100.00	
1	PRIMARY	Pass	ref	9	100.00	Using index
1	PRIMARY	Visit	ref	2	100.00	Using index
2	SUBQUERY	Pass	index	97617	100.00	Using index; Using temporary; Using...
2	SUBQUERY	Visit	ref	2	100.00	Using index

Рис. 33. Табличное представление последовательности выполнения запроса

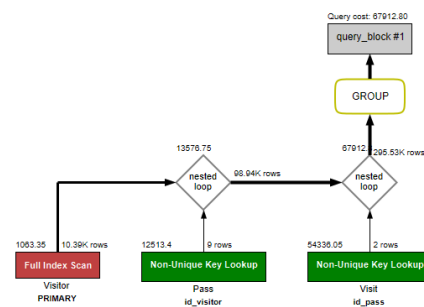


Рис. 34. Графическое представление последовательности выполнения запроса

фамилия	имя	отчество	количество посещений
Щегловитов	Евдокия	Мartiновна	65
Денисов	Милана	Игнатович	65

Рис. 35. Результат запроса 8.1

4.12 Запрос №8.2

Необходимо найти посетителей с минимальным числом посещений.

Графическое и табличное представление последовательности выполнения отчета представлено на рисунках 36 и 37.

Реляционная формула:

$$\pi \left(\begin{array}{l} \beta(\text{visitor.visitor_surname as «фамилия»}), \\ \beta(\text{visitor.visitor_name as «имя»}), \\ \beta(\text{visitor.visitor_patronymic as «отчество»}), \\ \beta(\text{count(visit.id_visit) as «количество посещений»}) \end{array} \right) \\
 \sigma(\text{количество посещений} = \min_visits) \\
 \text{Visitor} \bowtie_{(\text{visitor.id_pass} = \text{pass.id_pass})} \\
 \text{Pass} \bowtie_{(\text{pass.id_visitor} = \text{visit.id_visitor})} \text{Visit} \\
 \gamma(\text{id_visitor})$$

Листинг 11. Запрос 8.1

```

1 SELECT
2 visitor.visitor_surname as "фамилия",
3 visitor.visitor_name as "имя",
4 visitor.visitor_patronymic as "отчество",
5 COUNT(visit.id_visit) as "количество посещений"
6 FROM Visit
7 INNER JOIN Pass ON visit.id_pass = pass.id_pass
8 INNER JOIN Visitor ON pass.id_visitor = visitor.id_visitor
9 GROUP BY visitor.id_visitor
10 HAVING COUNT(visit.id_visit) = (
11 SELECT COUNT(visit.id_visit) as count_visit
12 FROM Visit
13 INNER JOIN Pass ON visit.id_pass = pass.id_pass
14 GROUP BY pass.id_visitor
15 ORDER BY count_visit ASC
16 LIMIT 1
17 );

```

В результирующей таблице было выведено 3 строки, время выполнения запроса составляет 0.610 секунд.

id	select_type	table	type	rows	filtered	Extra
1	PRIMARY	Visitor	index	10391	100.00	
1	PRIMARY	Pass	ref	9	100.00	Using index
1	PRIMARY	Visit	ref	2	100.00	Using index
2	SUBQUERY	Pass	index	97617	100.00	Using index; Using temporary; Using...
2	SUBQUERY	Visit	ref	2	100.00	Using index

Рис. 36. Табличное представление последовательности выполнения запроса

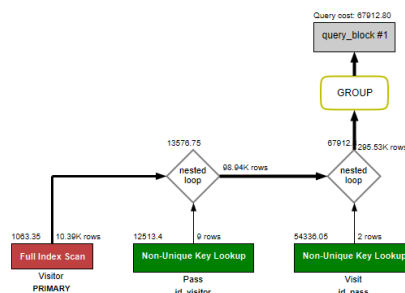


Рис. 37. Графическое представление последовательности выполнения запроса

фамилия	имя	отчество	количество посещений
Худякин	Капитолина	Захаровна	1
Коновалов	Марат	Игорьевич	1
Фролов	Мария	Викторович	1

Рис. 38. Результат запроса 8.2

Заключение

Работа была выполнена в MySQL Workbench 8.0 CE и командной строке Windows. Спроектирована и создана база данных для коммерческого бассейна, содержащая 14 таблиц. Для графического описания базы данных созданы ER-диаграмма, схема сущностей и графическое представление базы данных на двух языках.

Генерация значений для базы данных было произведено программой на языке Python 3.10 с использованием библиотеки pymysql. Распределение данных основано на равномерном распределении. Всего в таблицах базы данных содержатся 418238 значений.

Для тестирования базы данных было разработано 8 запросов, которые были написаны на языке SQL. Каждый запрос был проанализирован с помощью команд EXPLAIN и Visual EXPLAIN. Для трех запросов были построены гистограммы, подтверждающие корректность генерации.

Работа была выполнена на ноутбуке с процессором AMD Ryzen 5 4600H, 8 ГБ оперативной памяти и операционной системой Windows 11.

Список литературы

- [1] <https://www.swim-sport.ru/articles/vidi-basseinov/> (дата обращения 08.04.2025)
- [2] ГОСТ Р 57015–2016 Услуги населению. Услуги бассейнов. Общие требования. — Введ. 2017–07–01. — М. : Стандартиформ, 2016. — [Электронный ресурс]. — URL: триозон.рф/гост-р-57015-2016-услуги-населению-услуги-бассейнов-общие-требования (дата обращения: 08.04.2025).
- [3] <https://77.rospotrebnadzor.ru/index.php/san-epid/76-napрав/question/2742-2015-02-03-10-53-55> (дата обращения 08.04.2025)
- [4] <https://pymysql.readthedocs.io/en/latest/> (дата обращения 08.04.2025)
- [5] Дейт К. «Введение в системы базы данных. 8-е изд.». М.: Вильямс, 2005. 1328 стр.
- [6] Кузнецов С.Д. «Базы данных. Модели и языки». М.: БИНОМ-ПРЕСС, 2008. 488 стр.

Приложение А «Исходный код программы генерации схемы базы данных»

Листинг 12. Создание таблиц базы данных

```
1 CREATE SCHEMA IF NOT EXISTS 'swimming_pool' DEFAULT CHARACTER SET utf8;
2 USE 'swimming_pool';
3
4
5 CREATE TABLE IF NOT EXISTS 'swimming_pool'.'Type_of_the_day' (
6 'id_day' INT PRIMARY KEY NOT NULL AUTO_INCREMENT,
7 'type_of_day' VARCHAR(8) NOT NULL DEFAULT 'будний'
8 );
9
10 CREATE TABLE IF NOT EXISTS 'swimming_pool'.'Weekday' (
11 'id_weekday' INT PRIMARY KEY AUTO_INCREMENT,
12 'day_week' VARCHAR(11) NOT NULL
13 );
14
15 CREATE TABLE IF NOT EXISTS 'swimming_pool'.'Skilltest' (
16 'id_skill' INT PRIMARY KEY AUTO_INCREMENT,
17 'complexity' VARCHAR(7) NOT NULL
18 );
19
20 CREATE TABLE IF NOT EXISTS 'swimming_pool'.'Category_of_pass' (
21 'id_category' INT PRIMARY KEY AUTO_INCREMENT,
22 'type_age' VARCHAR(8) NOT NULL
23 );
24
25 CREATE TABLE IF NOT EXISTS 'swimming_pool'.'Responsible' (
26 'id_responsible' INT PRIMARY KEY AUTO_INCREMENT,
27 'responsible_surname' VARCHAR(40) NOT NULL,
28 'responsible_name' VARCHAR(15) NOT NULL,
29 'responsible_patronymic' VARCHAR(20),
30 'responsible_phone' VARCHAR(15) NOT NULL
31 );
32
33 CREATE TABLE IF NOT EXISTS 'swimming_pool'.'Admin' (
34 'id_admin' INT PRIMARY KEY AUTO_INCREMENT,
35 'admin_surname' VARCHAR(40) NOT NULL,
36 'admin_name' VARCHAR(15) NOT NULL,
37 'admin_patronymic' VARCHAR(20),
38 'admin_experience' INT NOT NULL DEFAULT 0
39 );
40
41 CREATE TABLE IF NOT EXISTS 'swimming_pool'.'Kind_of_pass' (
42 'id_kind_of_pass' INT PRIMARY KEY AUTO_INCREMENT,
43 'kind_name' VARCHAR(17) NOT NULL,
44 'id_category' INT NOT NULL,
45 FOREIGN KEY ('id_category')
46 REFERENCES 'swimming_pool'.'Category_of_pass'('id_category')
47 ON DELETE NO ACTION
48 ON UPDATE NO ACTION
49 );
50
```

```

51 CREATE TABLE IF NOT EXISTS 'swimming_pool'.'Schedule' (
52 'id_schedule' INT PRIMARY KEY AUTO_INCREMENT,
53 'id_day' INT NOT NULL,
54 'start_time' TIME NOT NULL,
55 'end_time' TIME NOT NULL,
56 FOREIGN KEY ('id_day')
57 REFERENCES 'swimming_pool'.'Type_of_the_day'(('id_day'))
58 ON DELETE NO ACTION
59 ON UPDATE NO ACTION
60 );
61
62 CREATE TABLE IF NOT EXISTS 'swimming_pool'.'Type_of_pass' (
63 'id_type_of_pass' INT PRIMARY KEY AUTO_INCREMENT,
64 'id_schedule' INT NOT NULL,
65 'days_session' VARCHAR(30) NOT NULL,
66 'id_kind_of_pass' INT NOT NULL,
67 FOREIGN KEY ('id_schedule')
68 REFERENCES 'swimming_pool'.'Schedule'(('id_schedule'))
69 ON DELETE NO ACTION
70 ON UPDATE NO ACTION,
71 FOREIGN KEY ('id_kind_of_pass')
72 REFERENCES 'swimming_pool'.'Kind_of_pass'(('id_kind_of_pass'))
73 ON DELETE NO ACTION
74 ON UPDATE NO ACTION
75 );
76
77 CREATE TABLE IF NOT EXISTS 'swimming_pool'.'Visitor' (
78 'id_visitor' INT PRIMARY KEY AUTO_INCREMENT,
79 'visitor_surname' VARCHAR(40) NOT NULL,
80 'visitor_name' VARCHAR(15) NOT NULL,
81 'visitor_patronymic' VARCHAR(20),
82 'visitor_date_of_birth' DATE NOT NULL,
83 'id_responsible' INT,
84 'medical_certificate' BOOLEAN NOT NULL DEFAULT FALSE,
85 'visitor_phone' VARCHAR(15) NOT NULL,
86 FOREIGN KEY ('id_responsible')
87 REFERENCES 'swimming_pool'.'Responsible'(('id_responsible'))
88 ON DELETE NO ACTION
89 ON UPDATE NO ACTION
90 );
91
92 CREATE TABLE IF NOT EXISTS 'swimming_pool'.'Pass' (
93 'id_pass' INT PRIMARY KEY AUTO_INCREMENT,
94 'id_type_of_pass' INT NOT NULL,
95 'pass_start_date' DATE NOT NULL,
96 'pass_finish_date' DATE NOT NULL,
97 'id_visitor' INT NOT NULL,
98 'pass_number_visits' INT NOT NULL DEFAULT 2,
99 FOREIGN KEY ('id_type_of_pass')
100 REFERENCES 'swimming_pool'.'Type_of_pass'(('id_type_of_pass'))
101 ON DELETE NO ACTION
102 ON UPDATE NO ACTION,
103 FOREIGN KEY ('id_visitor')
104 REFERENCES 'swimming_pool'.'Visitor'(('id_visitor'))
105 ON DELETE NO ACTION
106 ON UPDATE NO ACTION

```



```

107 );
108
109 CREATE TABLE IF NOT EXISTS 'swimming_pool'.'Price_list' (
110 'id_price_list' INT PRIMARY KEY AUTO_INCREMENT,
111 'cost' DECIMAL(10, 2) NOT NULL DEFAULT 0.00,
112 'id_skill' INT NOT NULL,
113 'lesson_volume' INT NOT NULL DEFAULT 1,
114 'id_type_of_pass' INT NOT NULL,
115 FOREIGN KEY ('id_skill')
116 REFERENCES 'swimming_pool'.'Skilltest'('id_skill')
117 ON DELETE NO ACTION
118 ON UPDATE NO ACTION,
119 FOREIGN KEY ('id_type_of_pass')
120 REFERENCES 'swimming_pool'.'Type_of_pass'('id_type_of_pass')
121 ON DELETE NO ACTION
122 ON UPDATE NO ACTION
123 );
124 select * from 'swimming_pool'.'Price_list';
125 CREATE TABLE IF NOT EXISTS 'swimming_pool'.'Purchase' (
126 'id_purchase' INT PRIMARY KEY AUTO_INCREMENT,
127 'purchase_date' DATE NOT NULL,
128 'id_admin' INT NOT NULL,
129 'id_pass' INT NOT NULL,
130 'id_visitor' INT NOT NULL,
131 'id_price_list' INT NOT NULL,
132 FOREIGN KEY ('id_admin')
133 REFERENCES 'swimming_pool'.'Admin'('id_admin')
134 ON DELETE NO ACTION
135 ON UPDATE NO ACTION,
136 FOREIGN KEY ('id_pass')
137 REFERENCES 'swimming_pool'.'Pass'('id_pass')
138 ON DELETE NO ACTION
139 ON UPDATE NO ACTION,
140 FOREIGN KEY ('id_visitor')
141 REFERENCES 'swimming_pool'.'Visitor'('id_visitor')
142 ON DELETE NO ACTION
143 ON UPDATE NO ACTION,
144 FOREIGN KEY ('id_price_list')
145 REFERENCES 'swimming_pool'.'Price_list'('id_price_list')
146 ON DELETE NO ACTION
147 ON UPDATE NO ACTION
148 );
149
150 CREATE TABLE IF NOT EXISTS 'swimming_pool'.'Visit' (
151 'id_visit' INT PRIMARY KEY AUTO_INCREMENT,
152 'visit_date' DATE NOT NULL,
153 'id_weekday' INT NOT NULL,
154 'id_schedule' INT NOT NULL,
155 'id_admin' INT NOT NULL,
156 'id_pass' INT NOT NULL,
157 FOREIGN KEY ('id_weekday')
158 REFERENCES 'swimming_pool'.'Weekday'('id_weekday')
159 ON DELETE NO ACTION
160 ON UPDATE NO ACTION,
161 FOREIGN KEY ('id_schedule')
162 REFERENCES 'swimming_pool'.'Schedule'('id_schedule')

```

```
163 ON DELETE NO ACTION
164 ON UPDATE NO ACTION,
165 FOREIGN KEY ('id_admin')
166 REFERENCES 'swimming_pool'.'Admin'('id_admin')
167 ON DELETE NO ACTION
168 ON UPDATE NO ACTION,
169 FOREIGN KEY ('id_pass')
170 REFERENCES 'swimming_pool'.'Pass'('id_pass')
171 ON DELETE NO ACTION
172 ON UPDATE NO ACTION
173 );
```

Приложение Б «Исходный код программы заполнения базы данных тестовыми данными»

Листинг 13. Заполнение таблицы Type_of_the_day

```
1 import pymysql
2
3 connection = pymysql.connect(
4 host='localhost',
5 user='root',
6 password='mysqlpassword79',
7 database='swimming_pool',
8 charset='utf8mb4'
9 )
10
11 try:
12 with connection.cursor() as cursor:
13     cursor.execute("""
14         INSERT INTO 'swimming_pool'.'Type_of_the_day' ('type_of_day')
15         VALUES
16         будний(''),
17         выходной('');
18     """)
19 connection.commit()
20 finally:
21 connection.close()
```

Листинг 14. Заполнение таблицы Weekday

```
1 import pymysql
2
3 connection = pymysql.connect(
4 host='localhost',
5 user='root',
6 password='mysqlpassword79',
7 database='swimming_pool',
8 charset='utf8mb4'
9 )
10
11 try:
12 with connection.cursor() as cursor:
13     cursor.execute("""
14         INSERT INTO 'swimming_pool'.'Weekday' ('day_week')
15         VALUES
16         понедельник(''),
17         вторник(''),
18         среда(''),
19         четверг(''),
20         пятница(''),
21         суббота(''),
22         воскресенье('');
23     """)
24 connection.commit()
25 finally:
26 connection.close()
```

Листинг 15. Заполнение таблицы Complexity

```

1  import pymysql
2
3  connection = pymysql.connect(
4  host='localhost',
5  user='root',
6  password='mysqlpassword79',
7  database='swimming_pool',
8  charset='utf8mb4'
9  )
10
11 try:
12 with connection.cursor() as cursor:
13     cursor.execute("""
14         INSERT INTO 'swimming_pool'.'Skilltest' ('complexity')
15         VALUES
16         легкий(''),
17         средний(''),
18         сложный('');
19     """)
20 connection.commit()
21 finally:
22 connection.close()

```

Листинг 16. Заполнение таблицы Category_of_pass

```

1  import pymysql
2
3  connection = pymysql.connect(
4  host='localhost',
5  user='root',
6  password='mysqlpassword79',
7  database='swimming_pool',
8  charset='utf8mb4'
9  )
10
11 try:
12 with connection.cursor() as cursor:
13     cursor.execute("""
14         INSERT INTO 'swimming_pool'.'Category_of_pass' ('type_age')
15         VALUES
16         детский(''),
17         взрослый('');
18     """)
19 connection.commit()
20 finally:
21 connection.close()

```

Листинг 17. Заполнение таблицы Kind_of_pass

```

1  import pymysql
2
3  connection = pymysql.connect(

```

```

4  host='localhost',
5  user='root',
6  password='mysqlpassword79',
7  database='swimming_pool',
8  charset='utf8mb4'
9  )
10
11 try:
12 with connection.cursor() as cursor:
13     cursor.execute("""
14     INSERT INTO 'swimming_pool'.'Kind_of_pass' ('kind_name', '
15     id_category')
16     VALUES
17     обучение(' плаванию', 1),
18     спортивное('', 1),
19     оздоровительное('', 1),
20     обучение(' плаванию', 2),
21     аквааэробика('', 2);
22     """)
23 connection.commit()
24 finally:
25 connection.close()

```

Листинг 18. Заполнение таблицы Schedule

```

1  import pymysql
2
3  connection = pymysql.connect(
4  host='localhost',
5  user='root',
6  password='mysqlpassword79',
7  database='swimming_pool',
8  charset='utf8mb4'
9  )
10
11 try:
12 with connection.cursor() as cursor:
13     cursor.execute("""
14     INSERT INTO 'swimming_pool'.'Schedule' ('id_day', 'start_time', '
15     end_time')
16     VALUES
17     (1, '8:00:00', '10:00:00'),
18     (1, '10:00:00', '12:00:00'),
19     (1, '12:00:00', '13:00:00'),
20     (1, '13:00:00', '14:00:00'),
21     (1, '15:00:00', '18:00:00'),
22     (1, '19:00:00', '21:00:00'),
23     (1, '21:00:00', '22:00:00'),
24     (2, '10:00:00', '11:00:00'),
25     (2, '11:00:00', '12:00:00'),
26     (2, '12:00:00', '13:00:00'),
27     (2, '13:00:00', '14:00:00'),
28     (2, '15:00:00', '17:00:00'),
29     (2, '17:00:00', '19:00:00'),
30     (2, '19:00:00', '21:00:00');
31     """)

```

```

31 connection.commit()
32 finally:
33 connection.close()

```

Листинг 19. Заполнение таблицы admin

```

1 import random
2 import pymysql
3
4 with open('familii.txt', 'r', encoding='utf-8') as f:
5 surnames = [line.strip() for line in f if line.strip()]
6 with open('imena.txt', 'r', encoding='utf-8') as f:
7 names = [line.strip() for line in f if line.strip()]
8 with open('otchestva.txt', 'r', encoding='utf-8') as f:
9 patronymics = [line.strip() for line in f if line.strip()]
10
11 connection = pymysql.connect(
12 host='localhost',
13 user='root',
14 password='mysqlpassword79',
15 database='swimming_pool',
16 charset='utf8mb4'
17 )
18
19 try:
20 with connection.cursor() as cursor:
21 for _ in range(50):
22 admin_surname = random.choice(surnames)
23 admin_name = random.choice(names)
24 admin_patronymic = random.choice(patronymics)
25 admin_experience = random.randint(1, 15)
26
27 sql = """INSERT INTO 'Admin'
28 ('admin_surname', 'admin_name', 'admin_patronymic', 'admin_experience')
29 VALUES (%s, %s, %s, %s)"""
30 cursor.execute(sql, (admin_surname, admin_name, admin_patronymic,
31 admin_experience))
32 connection.commit()
33 finally:
34 connection.close()

```

Листинг 20. Заполнение таблицы responsible

```

1 import random
2 import pymysql
3
4 def generate_phone():
5 num = random.randint(90000010000, 9999999999)
6 return f"8{num}"
7
8 with open('familii.txt', 'r', encoding='utf-8') as f:
9 surnames = [line.strip() for line in f if line.strip()]
10 with open('imena.txt', 'r', encoding='utf-8') as f:
11 names = [line.strip() for line in f if line.strip()]
12 with open('otchestva.txt', 'r', encoding='utf-8') as f:

```

```

13 patronymics = [line.strip() for line in f if line.strip()]
14
15 connection = pymysql.connect(
16     host='localhost',
17     user='root',
18     password='mysqlpassword79',
19     database='swimming_pool',
20     charset='utf8mb4'
21 )
22
23 try:
24     with connection.cursor() as cursor:
25         for _ in range(5000):
26             responsible_surname = random.choice(surnames)
27             responsible_name = random.choice(names)
28             responsible_patronymic = random.choice(patronymics)
29             responsible_phone = generate_phone()
30
31             sql = """INSERT INTO 'Responsible'
32             ('responsible_surname', 'responsible_name', 'responsible_patronymic',
33              'responsible_phone')
34             VALUES (%s, %s, %s, %s)"""
35             cursor.execute(sql, (responsible_surname, responsible_name,
36                                 responsible_patronymic, responsible_phone))
37
38         connection.commit()
39     finally:
40         connection.close()

```

Листинг 21. Заполнение таблицы type_of_pass

```

1 import random
2 import pymysql
3
4 connection = pymysql.connect(
5     host='localhost',
6     user='root',
7     password='mysqlpassword79',
8     database='swimming_pool',
9     charset='utf8mb4'
10 )
11
12 weekday_names = {
13     1: "понедельник",
14     2: "вторник",
15     3: "среда",
16     4: "четверг",
17     5: "пятница",
18     6: "суббота",
19     7: "воскресенье"
20 }
21
22 try:
23     with connection.cursor() as cursor:
24         cursor.execute("SELECT id_schedule FROM Schedule")

```

```

25 schedule_ids = [row[0] for row in cursor.fetchall()]
26 cursor.execute("SELECT id_kind_of_pass FROM Kind_of_pass")
27 kind_ids = [row[0] for row in cursor.fetchall()]
28 weekday_groups = {
29     'будние': [1, 2, 3, 4, 5],
30     'выходные': [6, 7]
31 }
32
33 for i in range(1, 201):
34     id_schedule = random.choice(schedule_ids)
35     id_kind_of_pass = random.choice(kind_ids)
36     is_weekday = random.choice([True, False])
37     selected_day_ids = weekday_groups['будние'] if is_weekday else
38         weekday_groups['выходные']
39     max_days = len(selected_day_ids)
40     days_count = random.randint(1, min(3, max_days))
41     selected_day_ids = random.sample(selected_day_ids, days_count)
42     selected_days = [weekday_names[day_id] for day_id in selected_day_ids]
43     days_session = ",".join(selected_days)
44
45     cursor.execute(
46         """INSERT INTO Type_of_pass
47         (id_schedule, days_session, id_kind_of_pass)
48         VALUES (%s, %s, %s)""",
49         (id_schedule, days_session, id_kind_of_pass)
50     )
51     connection.commit()
52 finally:
53     connection.close()

```

Листинг 22. Заполнение таблицы visitor

```

1 import random
2 import pymysql
3 from datetime import datetime, timedelta
4
5 def generate_phone():
6     num = random.randint(9000010000, 9999999999)
7     return f"8{num}"
8
9 def generate_birth_date(is_child):
10     now = datetime.now()
11     if is_child:
12         end_date = now - timedelta(days=365*3)
13         start_date = now - timedelta(days=365*18)
14     else:
15         end_date = now - timedelta(days=365*18)
16         start_date = now - timedelta(days=365*90)
17
18     random_days = random.randint(0, (end_date - start_date).days)
19     return (start_date + timedelta(days=random_days)).strftime('%Y-%m-%d')
20
21 with open('familii.txt', 'r', encoding='utf-8') as f:
22     surnames = [line.strip() for line in f if line.strip()]
23 with open('imena.txt', 'r', encoding='utf-8') as f:

```



```

24 names = [line.strip() for line in f if line.strip()]
25 with open('otchestva.txt', 'r', encoding='utf-8') as f:
26     patronymics = [line.strip() for line in f if line.strip()]
27
28     connection = pymysql.connect(
29     host='localhost',
30     user='root',
31     password='mysqlpassword79',
32     database='swimming_pool',
33     charset='utf8mb4'
34     )
35
36     try:
37     with connection.cursor() as cursor:
38     cursor.execute("SELECT id_responsible FROM Responsible")
39     responsible_ids = [row[0] for row in cursor.fetchall()]
40     responsible_children = {rid: 0 for rid in responsible_ids}
41
42     for i in range(10400):
43     is_child = i < 5200
44     visitor_surname = random.choice(surnames)
45     visitor_name = random.choice(names)
46     visitor_patronymic = random.choice(patronymics) if random.random() > 0.1
47     else None
48     visitor_date_of_birth = generate_birth_date(is_child)
49     medical_certificate = random.choice([True, False]) if is_child else True
50     visitor_phone = generate_phone()
51
52     if is_child:
53     available_responsible = [rid for rid, count in responsible_children.
54                             items() if count == 0]
55     if not available_responsible:
56     available_responsible = [rid for rid, count in responsible_children.
57                             items() if count < 5]
58     id_responsible = random.choice(available_responsible)
59     responsible_children[id_responsible] += 1
60     else:
61     id_responsible = None
62
63     sql = """INSERT INTO 'Visitor'
64     ('visitor_surname', 'visitor_name', 'visitor_patronymic',
65     'visitor_date_of_birth', 'id_responsible',
66     'medical_certificate', 'visitor_phone')
67     VALUES (%s, %s, %s, %s, %s, %s, %s)"""
68     cursor.execute(sql, (visitor_surname, visitor_name, visitor_patronymic,
69     visitor_date_of_birth, id_responsible,
70     medical_certificate, visitor_phone))
71     connection.commit()
72     finally:
73     connection.close()

```

Листинг 23. Заполнение таблицы pass

```

1     import random
2     import pymysql
3     from datetime import datetime, timedelta

```

```

4
5 connection = pymysql.connect(
6     host='localhost',
7     user='root',
8     password='mysqlpassword79',
9     database='swimming_pool',
10    charset='utf8mb4'
11 )
12
13 def generate_pass_dates():
14     start_date = datetime(2012, 1, 1)
15     end_date = datetime.now()
16     random_days = random.randint(0, (end_date-start_date).days)
17     pass_start = start_date +timedelta(days = random_days)
18     pass_finish = pass_start + timedelta(days=180)
19     return pass_start.date(), pass_finish.date()
20
21 try:
22     with connection.cursor() as cursor:
23         cursor.execute("SELECT id_type_of_pass FROM Type_of_pass")
24         type_of_pass_ids = [row[0] for row in cursor.fetchall()]
25         cursor.execute("SELECT id_visitor FROM Visitor")
26         visitor_ids = [row[0] for row in cursor.fetchall()]
27
28         for i in range(1, 99999):
29             id_type_of_pass = random.choice(type_of_pass_ids)
30             id_visitor = random.choice(visitor_ids)
31             pass_start_date, pass_finish_date = generate_pass_dates()
32             pass_number_visits = random.randint(1, 10)
33
34             cursor.execute(
35                 """INSERT INTO Pass
36                 (id_type_of_pass, pass_start_date, pass_finish_date, id_visitor,
37                  pass_number_visits)
38                 VALUES (%s, %s, %s, %s, %s)""",
39                 (id_type_of_pass, pass_start_date, pass_finish_date, id_visitor,
40                  pass_number_visits)
41             )
42             connection.commit()
43         finally:
44             connection.close()

```

Листинг 24. Заполнение таблицы price_list

```

1 import random
2 import pymysql
3
4 connection = pymysql.connect(
5     host='localhost',
6     user='root',
7     password='mysqlpassword79',
8     database='swimming_pool',
9     charset='utf8mb4'
10 )
11

```

```

12 def calculate_cost(skill_level, lesson_volume):
13     base_price = 400
14     skill_multiplier = {1: 1.0, 2: 1.5, 3: 2.0}
15     volume_step = 100
16     price = base_price + (lesson_volume - 1) * volume_step
17     price *= skill_multiplier[skill_level]
18     return round(price, 2)
19
20 try:
21     with connection.cursor() as cursor:
22         cursor.execute("SELECT id_skill FROM Skilltest")
23         skill_ids = [row[0] for row in cursor.fetchall()]
24
25         cursor.execute("SELECT id_type_of_pass FROM Type_of_pass")
26         type_of_pass_ids = [row[0] for row in cursor.fetchall()]
27
28         for type_of_pass_id in type_of_pass_ids:
29             num_records = random.randint(1, 3)
30             selected_skills = random.sample(skill_ids, min(num_records, len(
31                 skill_ids)))
32             for skill_id in selected_skills:
33                 skill_level = random.randint(1, 3)
34                 lesson_volume = random.randint(1, 10)
35                 cost = calculate_cost(skill_level, lesson_volume)
36
37                 cursor.execute(
38                     """INSERT INTO Price_list
39                     (cost, id_skill, lesson_volume, id_type_of_pass)
40                     VALUES (%s, %s, %s, %s)""",
41                     (cost, skill_id, lesson_volume, type_of_pass_id)
42                 )
43
44             connection.commit()
45         finally:
46             connection.close()

```

Листинг 25. Заполнение таблицы visits

```

1 import random
2 import pymysql
3 from datetime import datetime, timedelta
4
5 connection = pymysql.connect(
6     host='localhost',
7     user='root',
8     password='mysqlpassword79',
9     database='swimming_pool',
10    charset='utf8mb4'
11 )
12
13 try:
14     with connection.cursor() as cursor:
15         cursor.execute("SELECT id_weekday FROM Weekday")
16         weekday_ids = [row[0] for row in cursor.fetchall()]
17         cursor.execute("SELECT id_schedule FROM Schedule")

```

```

18 schedule_ids = [row[0] for row in cursor.fetchall()]
19 cursor.execute("SELECT id_admin FROM Admin")
20 admin_ids = [row[0] for row in cursor.fetchall()]
21 cursor.execute("SELECT id_pass FROM Pass")
22 pass_ids = [row[0] for row in cursor.fetchall()]
23
24 start_date = datetime(2012, 1, 1)
25 end_date = datetime.now()
26 date_range = (end_date - start_date).days
27
28 for pass_id in pass_ids:
29     num_visits = random.randint(1, 4)
30
31     for _ in range(num_visits):
32         random_days = random.randint(0, date_range)
33         visit_date = start_date + timedelta(days=random_days)
34         weekday = visit_date.weekday() + 1
35         id_weekday = weekday
36         id_schedule = random.choice(schedule_ids)
37         id_admin = random.choice(admin_ids)
38         cursor.execute(
39             """INSERT INTO Visit
40             (visit_date, id_weekday, id_schedule, id_admin, id_pass)
41             VALUES (%s, %s, %s, %s, %s)""" ,
42             (visit_date.strftime('%Y-%m-%d'),
43              id_weekday,
44              id_schedule,
45              id_admin,
46              pass_id)
47         )
48
49 connection.commit()
50 finally:
51     connection.close()

```

Листинг 26. Заполнение таблицы purchase

```

1 import random
2 import pymysql
3 from datetime import datetime, timedelta
4
5 connection = pymysql.connect(
6     host='localhost',
7     user='root',
8     password='mysqlpassword79',
9     database='swimming_pool',
10    charset='utf8mb4'
11 )
12
13 try:
14     with connection.cursor() as cursor:
15         cursor.execute("SELECT id_admin FROM Admin")
16         admin_ids = [row[0] for row in cursor.fetchall()]
17         cursor.execute("SELECT id_visitor FROM Visitor")
18         visitor_ids = [row[0] for row in cursor.fetchall()]
19         cursor.execute("SELECT id_pass FROM Pass")

```

```

20 pass_ids = [row[0] for row in cursor.fetchall()]
21 cursor.execute("SELECT id_price_list FROM Price_list")
22 price_list_ids = [row[0] for row in cursor.fetchall()]
23
24 start_date = datetime(2012, 1, 1)
25 end_date = datetime.now()
26 date_range = (end_date - start_date).days
27
28 for visitor_id in visitor_ids:
29     num_purchases = random.randint(3, 7)
30
31     for _ in range(num_purchases):
32         random_days = random.randint(0, date_range)
33         purchase_date = start_date + timedelta(days=random_days)
34         id_admin = random.choice(admin_ids)
35         id_pass = random.choice(pass_ids)
36         id_price_list = random.choice(price_list_ids)
37         cursor.execute(
38             """INSERT INTO Purchase
39 (purchase_date, id_admin, id_pass, id_visitor, id_price_list)
40 VALUES (%s, %s, %s, %s, %s)""",
41         (purchase_date.strftime('%Y-%m-%d'),
42          id_admin,
43          id_pass,
44          visitor_id,
45          id_price_list)
46         )
47
48     connection.commit()
49 finally:
50     connection.close()

```

Приложение 1. Список разрешенных предметов

1. Пояса для плавания.
2. Тренировочные маски.
3. Утяжелители для ног.
4. Утяжелители для рук.
5. Водные гантели.
6. Резиновые ленты.
7. Тренировочные канаты.
8. Жилеты-утяжелители.
9. Зажимы для носа.
10. Тренировочные трубки для дыхания.
11. Перчатки с перепонками.
12. Плавательные парашюты диаметром не более метра.